

AWS IAM Role-Based Access Control

IAM Users, Groups, Policies and EC2 Instance Roles with S3 Access

Project Documentation

1. Project Overview

This project demonstrates AWS Identity and Access Management using two IAM users, two IAM groups, managed policies, and EC2 instance roles to implement and validate role-based access control across Amazon EC2 and Amazon S3.

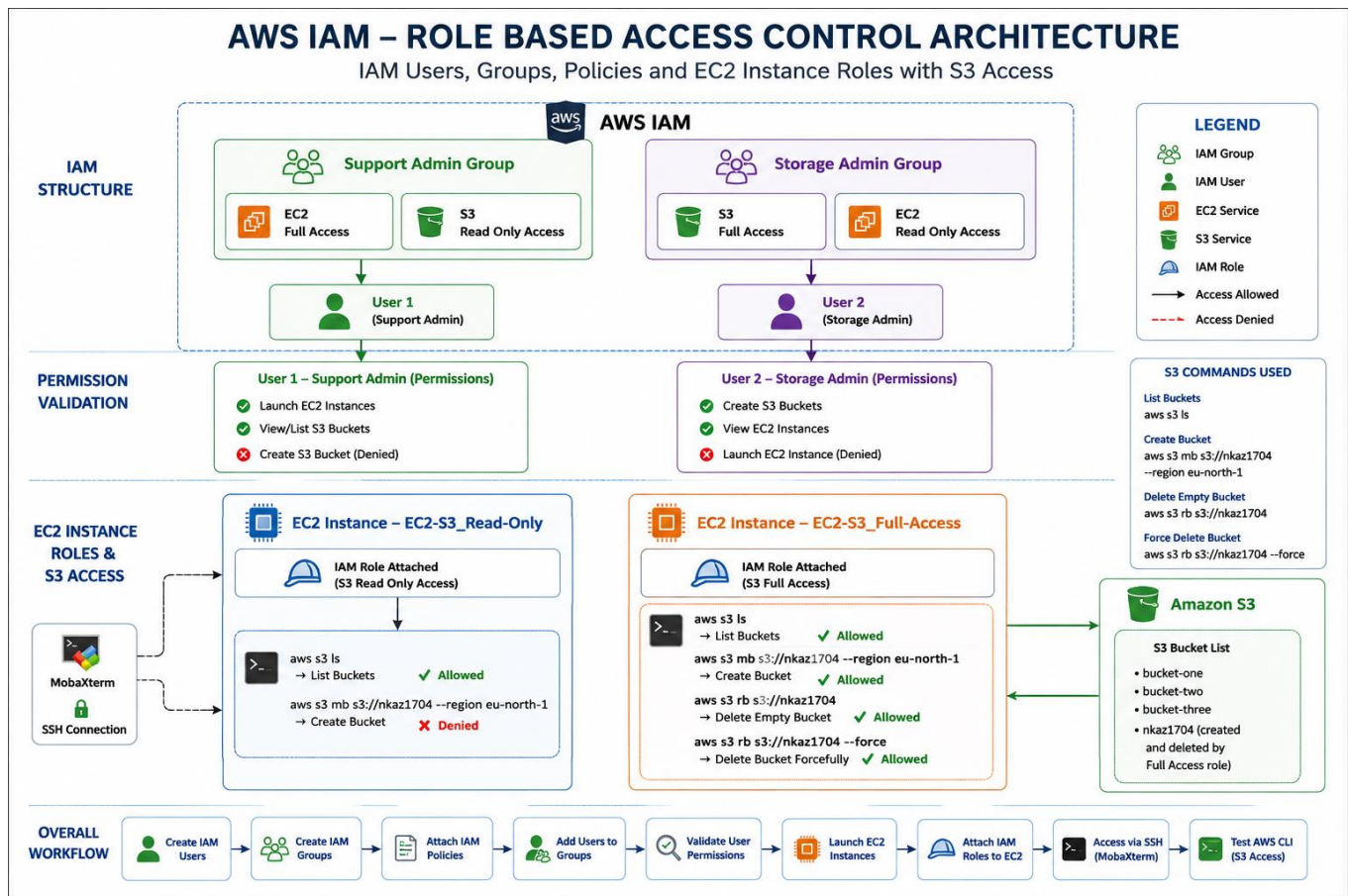
2. Objective

To understand how IAM users, groups, policies, and EC2 roles control access to AWS resources by validating both permitted and denied EC2 and S3 operations.

3. Technology Stack

- AWS IAM
- Amazon EC2
- Amazon S3
- AWS CLI
- MobaXterm
- IAM Roles and Policies
- Amazon Linux

4. Architecture



The architecture separates user permissions through IAM groups and instance permissions through IAM roles, enabling controlled EC2 and S3 access based on assigned policies.

5. Implementation Workflow

1. Create the Support Admin and Storage Admin IAM groups.
2. Create two IAM users and assign each user to the appropriate group.
3. Attach EC2 and S3 managed policies based on the required permission model.
4. Validate allowed and denied actions for both IAM users.
5. Launch two EC2 instances for IAM role-based access testing.
6. Create and configure separate IAM roles for S3 Read Only and S3 Full Access.
7. Attach the IAM roles to the corresponding EC2 instances.
8. Connect to both instances through SSH using MobaXterm.
9. Use AWS CLI commands to validate S3 access and permission restrictions.

6. Implementation Evidence & Step-by-Step Documentation

1. IAM User Group Creation

Created the initial IAM user group structure for organizing permissions by job responsibility.

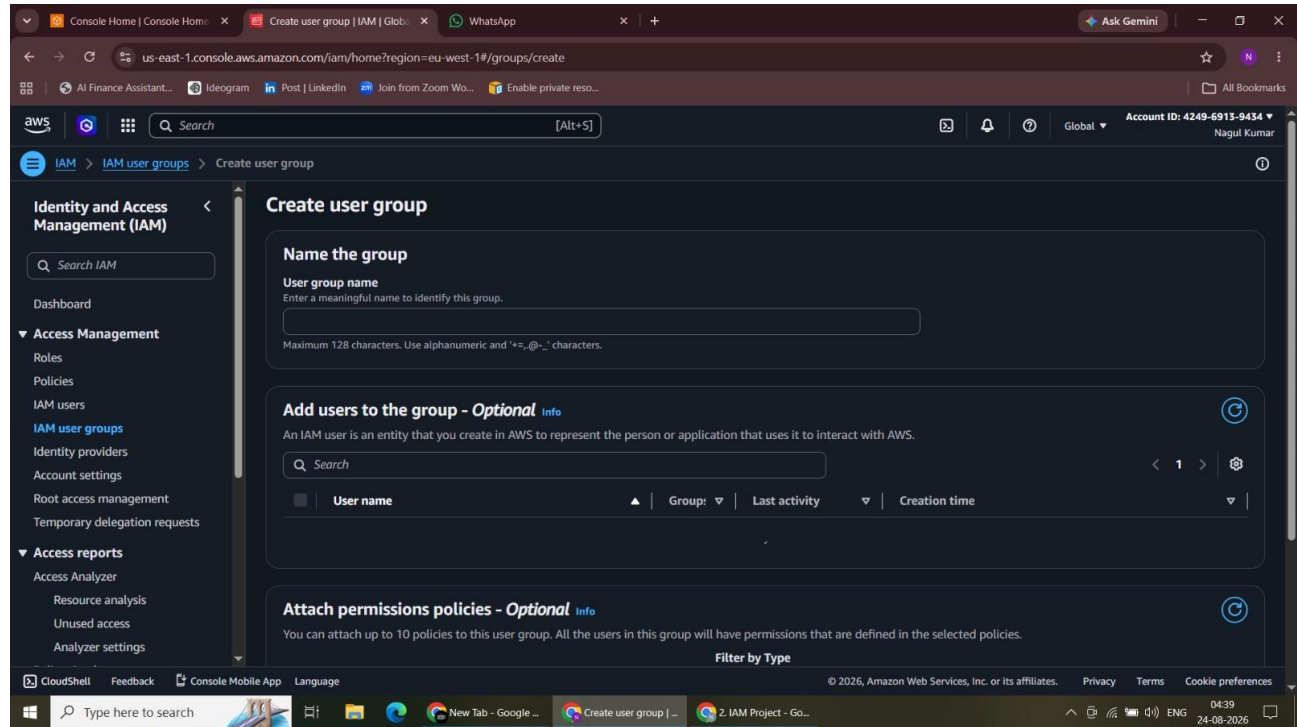


Figure 1. IAM User Group Creation

2. Support Admin Group Creation

Created the Support Admin IAM group to manage EC2 and S3 read-only permissions.

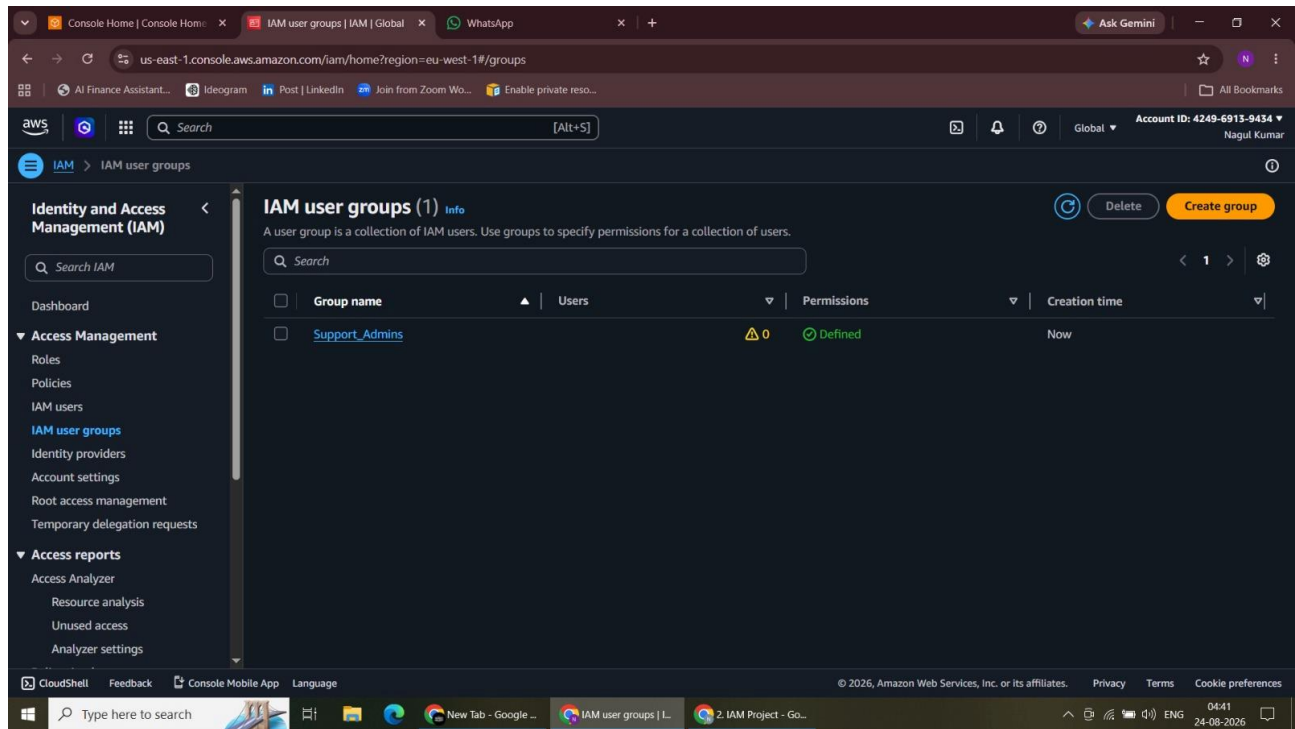


Figure 2.

Support Admin Group Creation

3. Storage Admin Group Creation

Created the Storage Admin IAM group to manage S3 full access and EC2 read-only permissions.

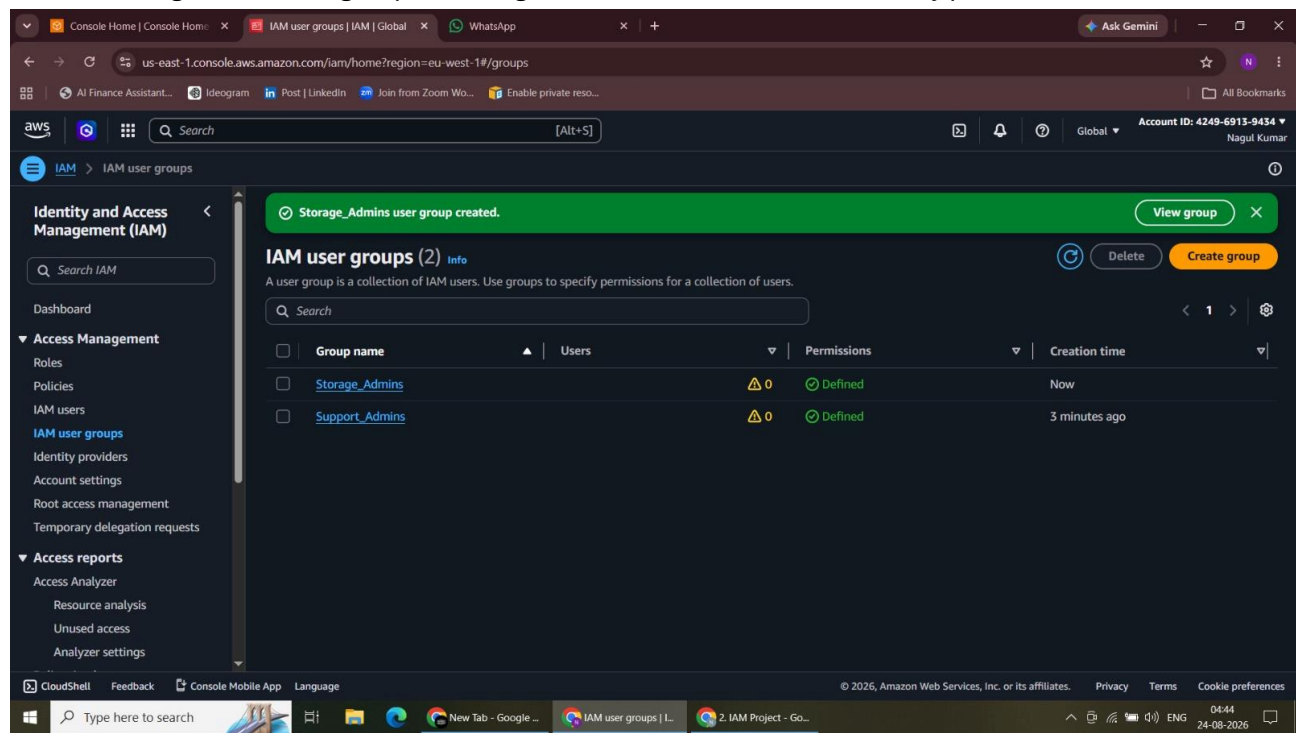


Figure 3. Storage Admin Group Creation

4. User 1 Creation

Configured and created User 1 as an IAM identity for Support Admin access validation.

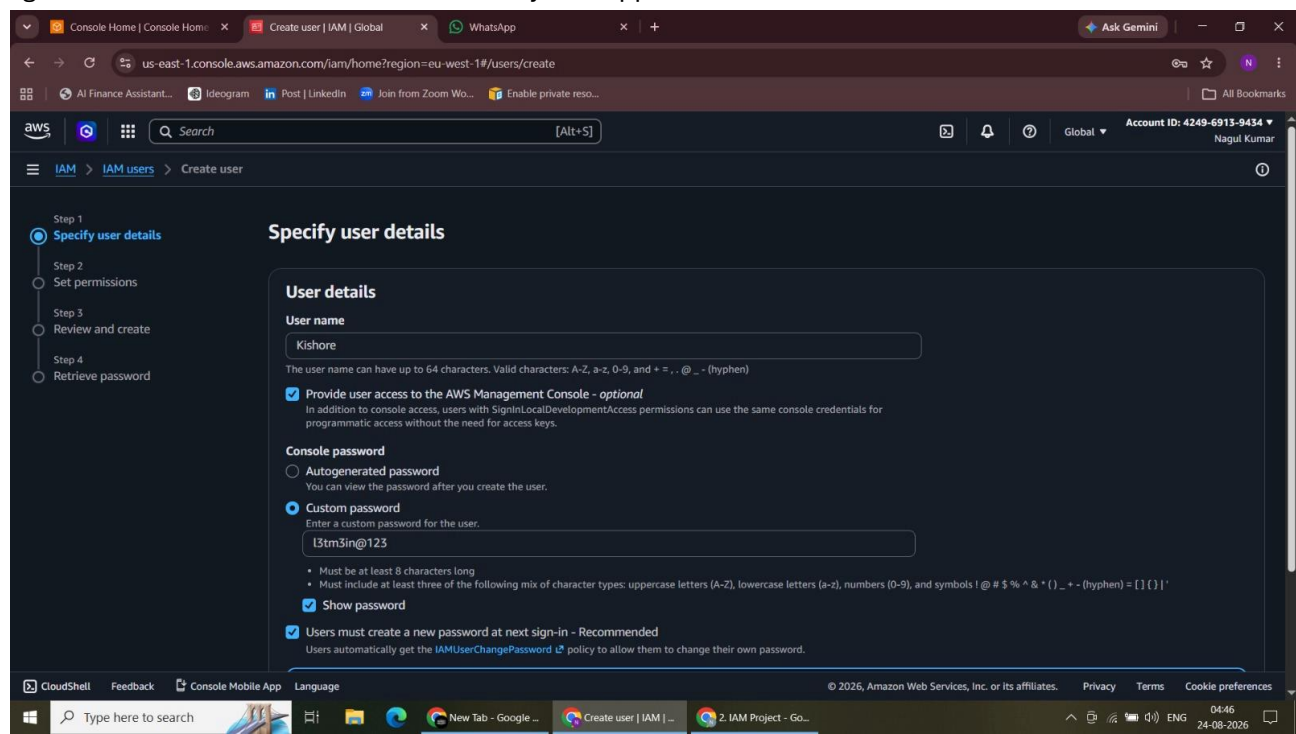


Figure 4. User 1 Creation

5. User 1 Group Assignment

Assigned User 1 to the Support Admin group so permissions are inherited through group policies.

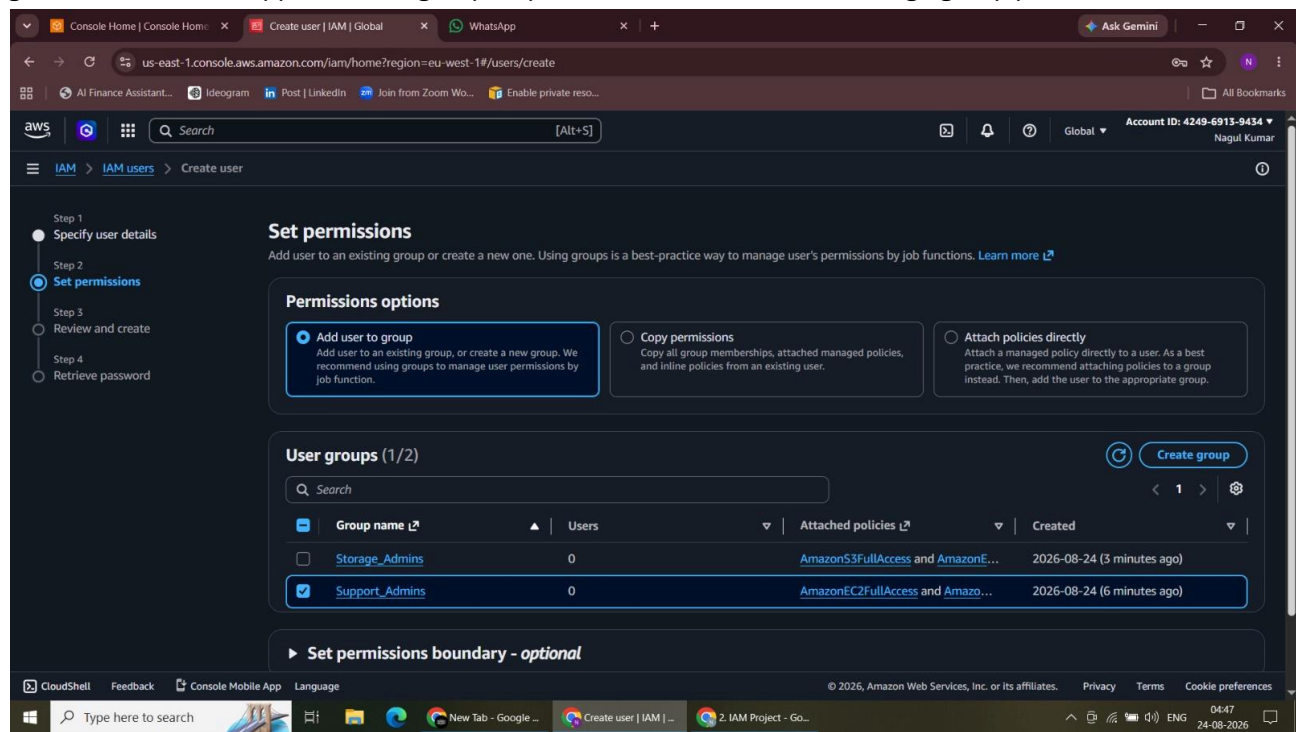


Figure 5. User 1 Group Assignment

6. User 1 Account Creation Confirmation

Completed the User 1 creation workflow and confirmed the IAM account setup.

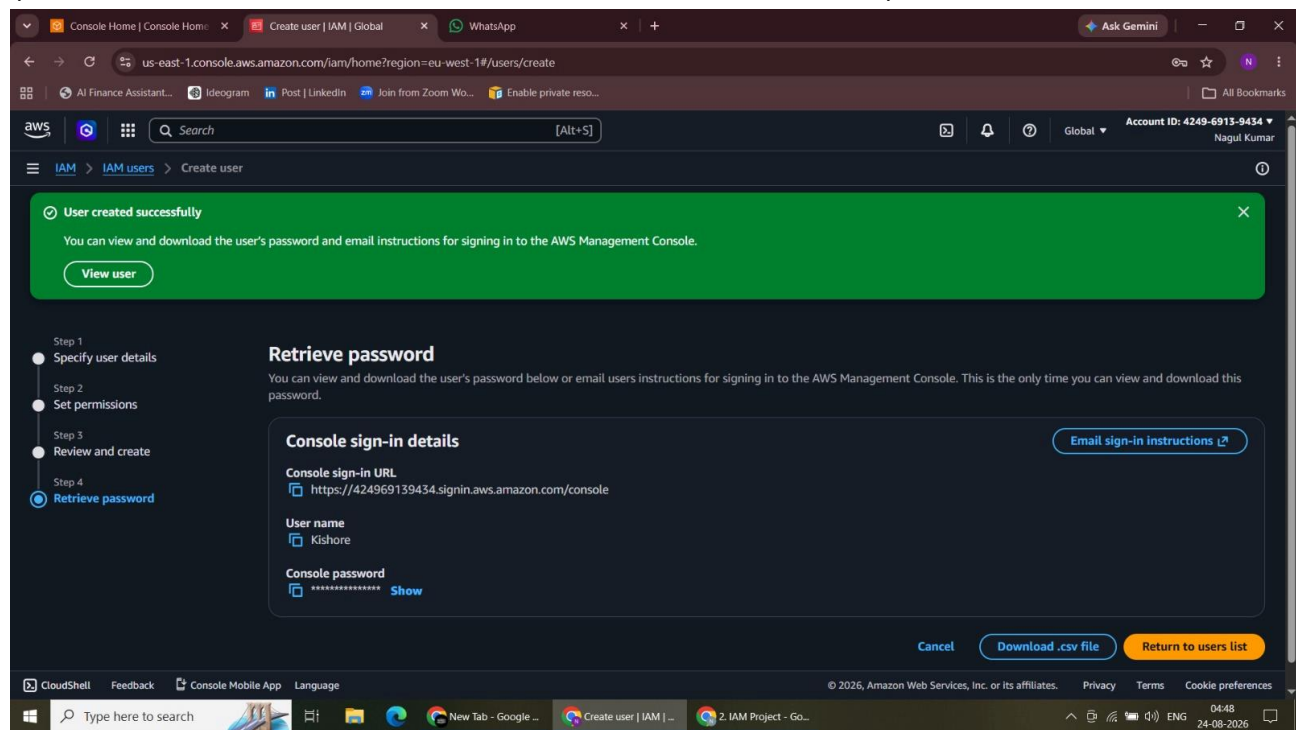


Figure 6. User 1 Account Creation Confirmation

7. User 1 Sign-In Credentials

Retrieved the IAM console sign-in credentials required for User 1 to access AWS.

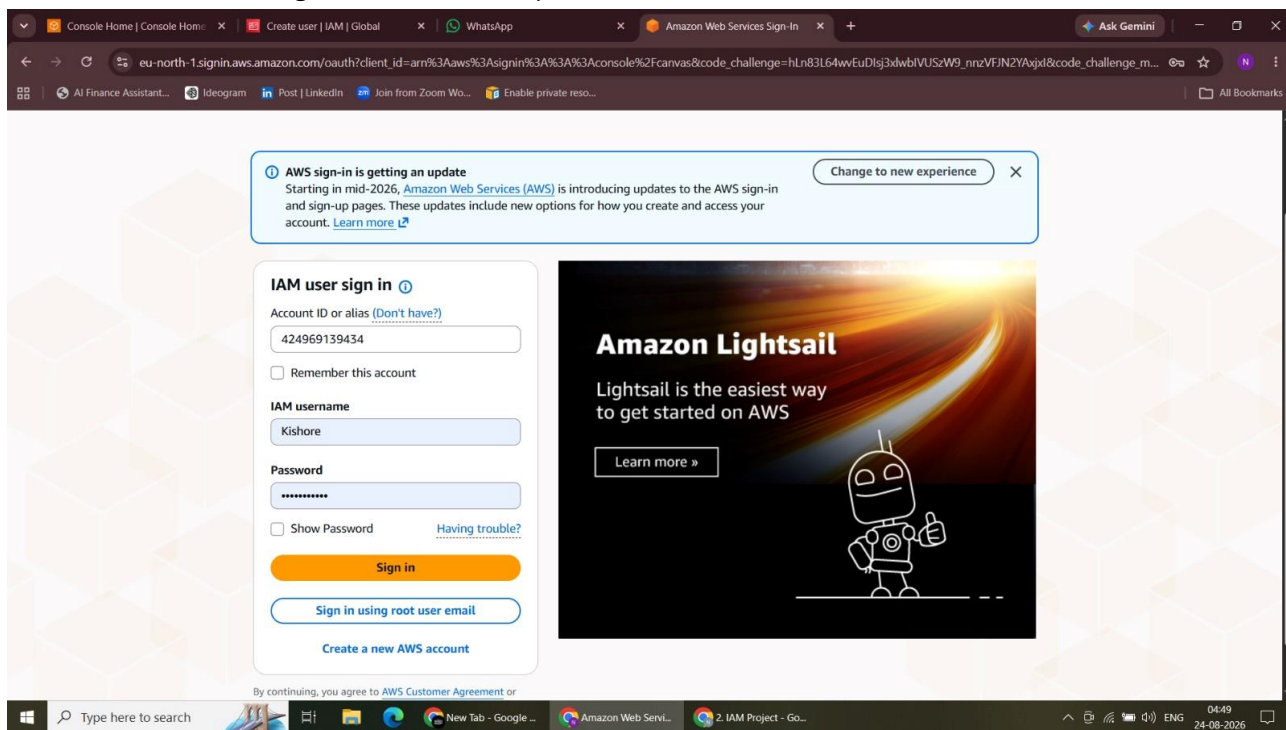


Figure 7. User 1 Sign-In Credentials

8. User 1 Sign-In and S3 Permission Test

Signed in as User 1 and tested the S3 workflow under the assigned read-only permissions.

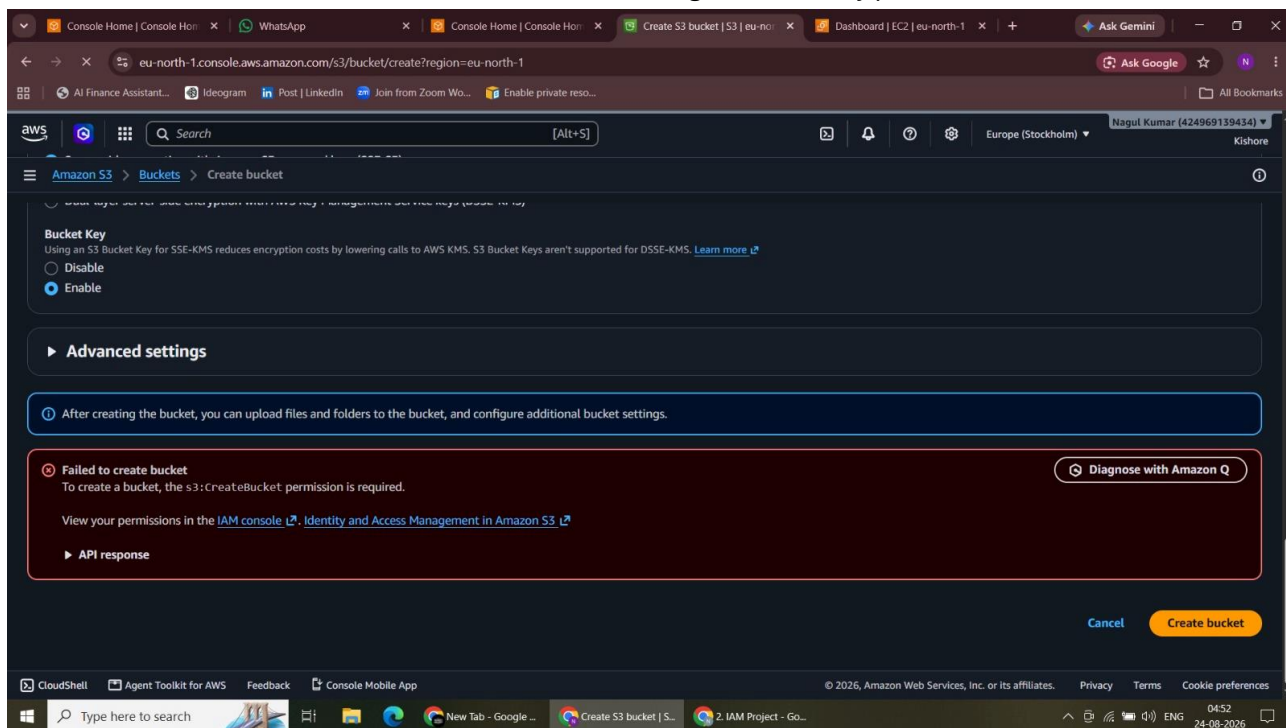


Figure 8. User 1 Sign-In and S3 Permission Test

9. EC2 Instance Launch by User 1

Used User 1's EC2 permissions to launch instances for IAM role-based access testing.

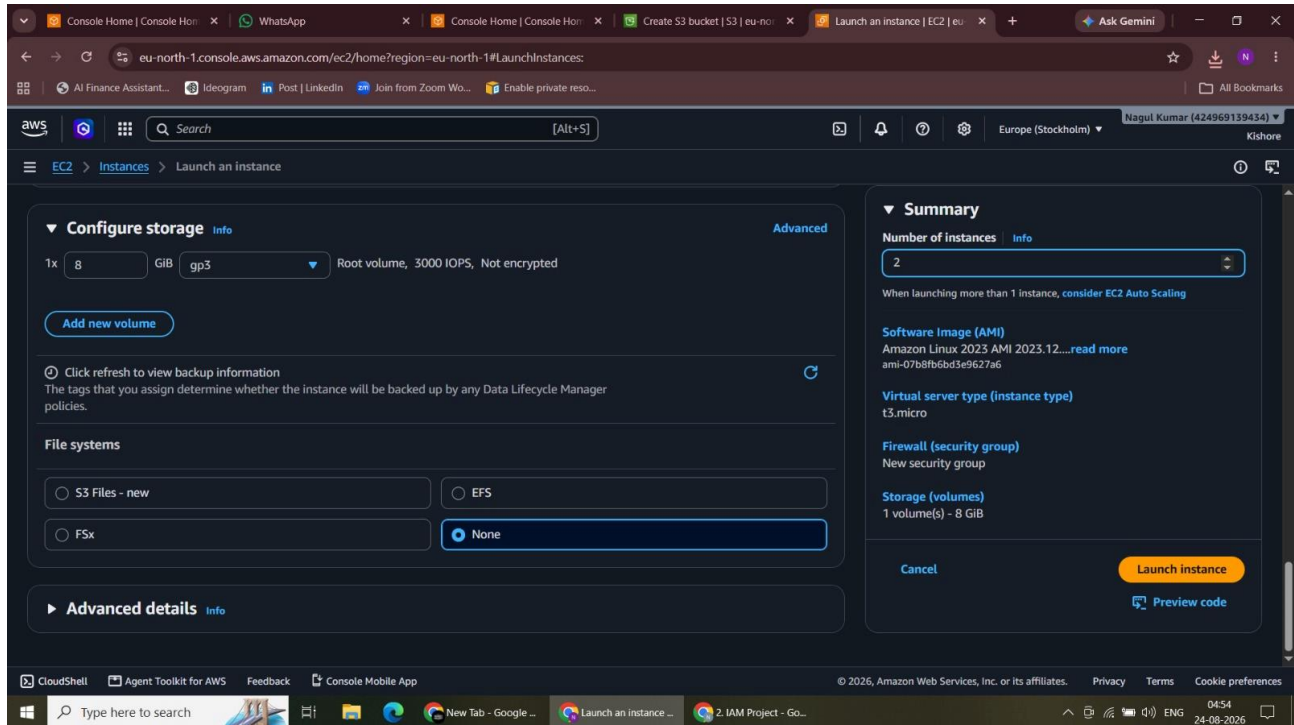


Figure 9. EC2 Instance Launch by User 1

10. Launched EC2 Instances Verification

Verified the EC2 instances created for testing role-based S3 access.

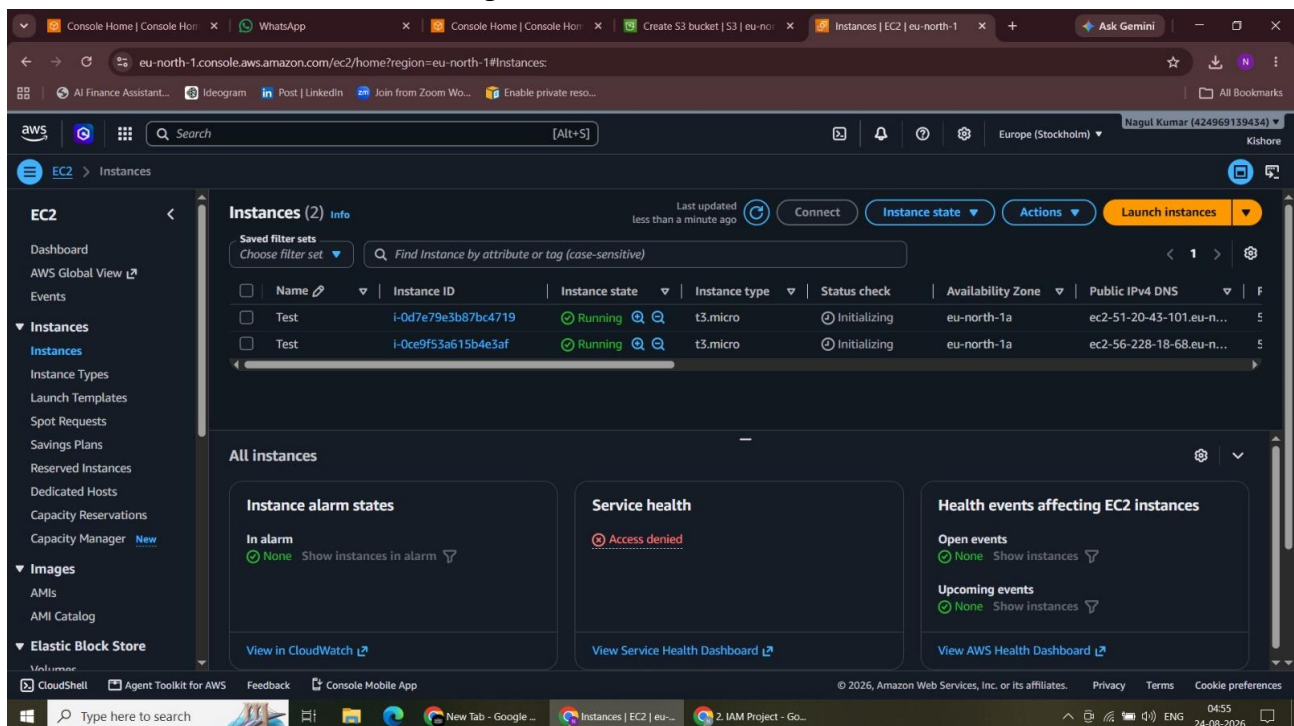


Figure 10. Launched EC2 Instances Verification

11. User Permission Assignment

Configured IAM user permissions through group membership rather than direct user policies.

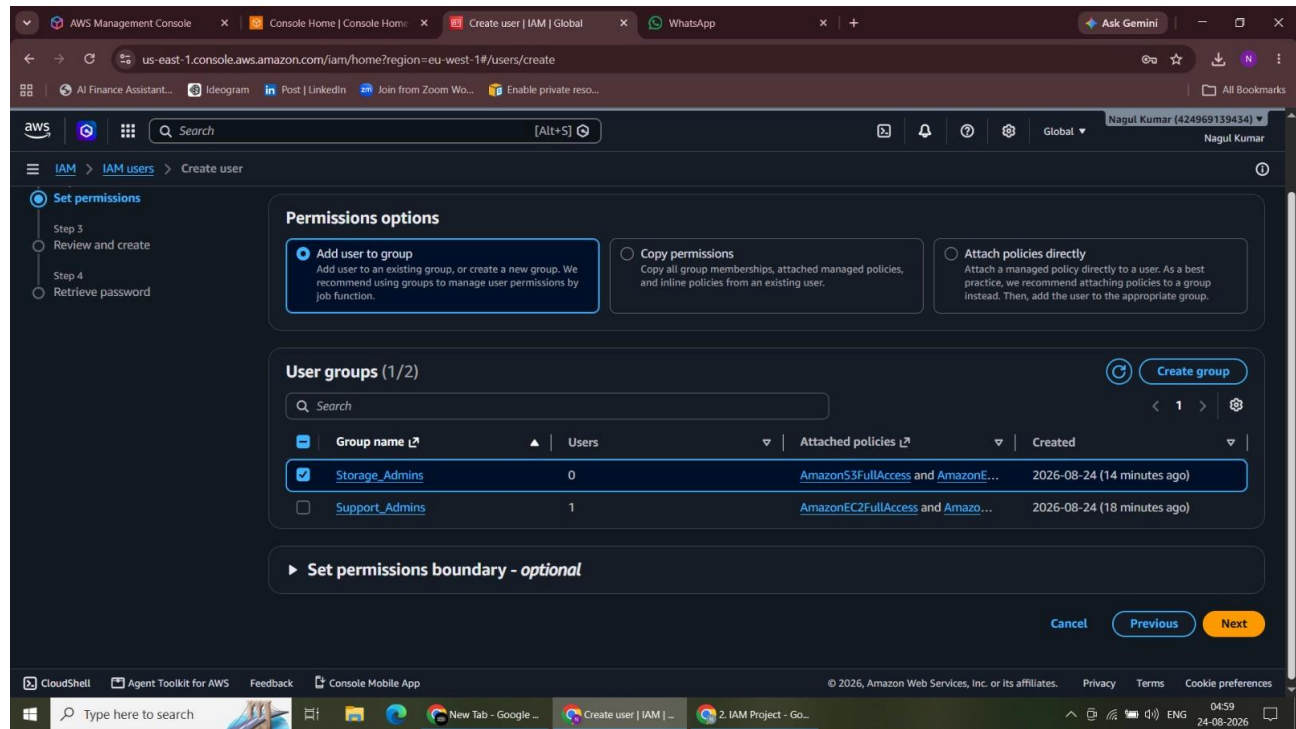


Figure 11. User Permission Assignment

12. User Permission Validation

Validated the assigned access model while completing the IAM user configuration workflow.

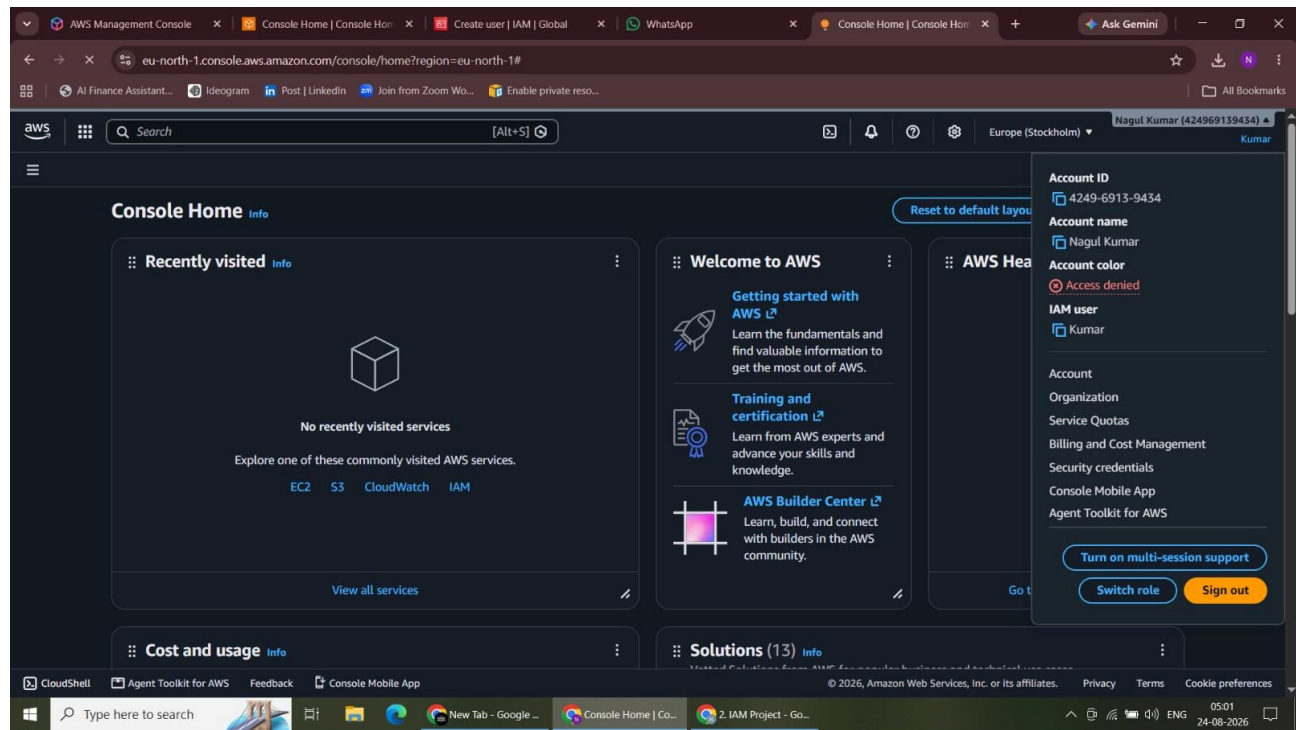


Figure 12. User Permission Validation

13. User 2 Console Sign-In

Signed in using the second IAM user account to test the Storage Admin permission set.

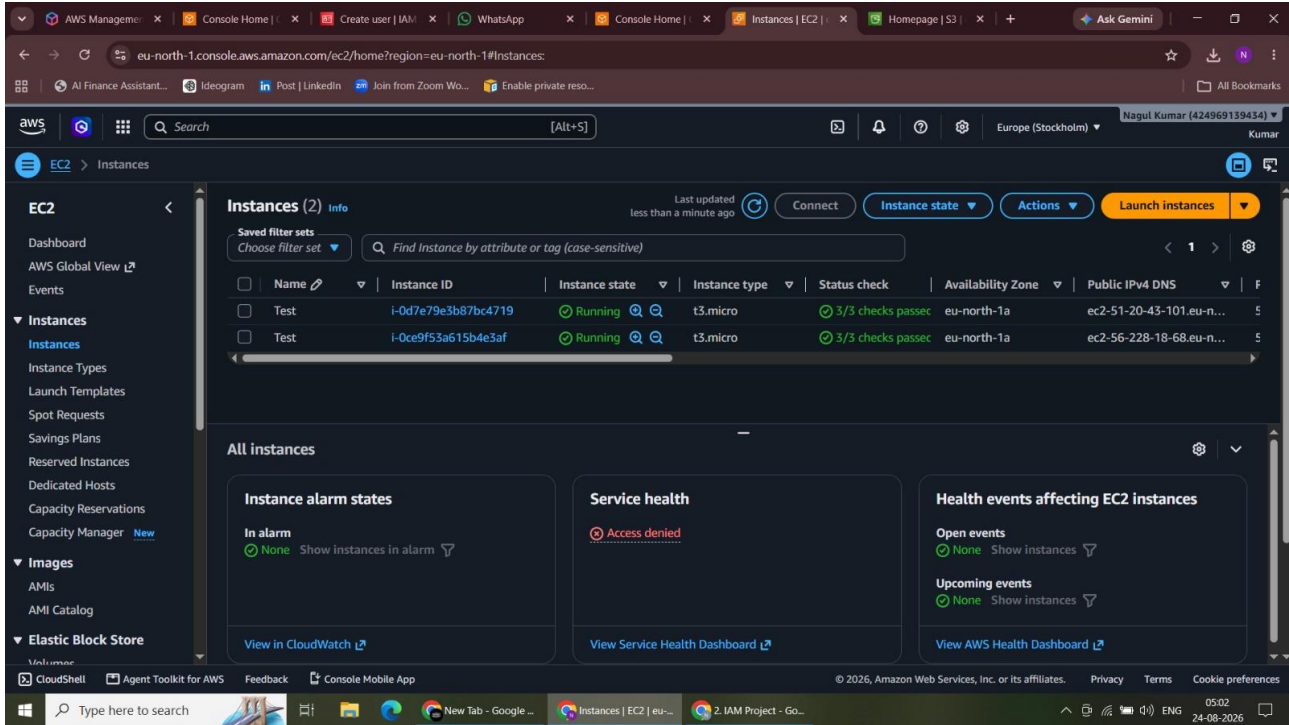


Figure 13. User 2 Console Sign-In

14. User 2 S3 Bucket Access Test

Tested the S3 bucket workflow using the permissions inherited by the Storage Admin user.

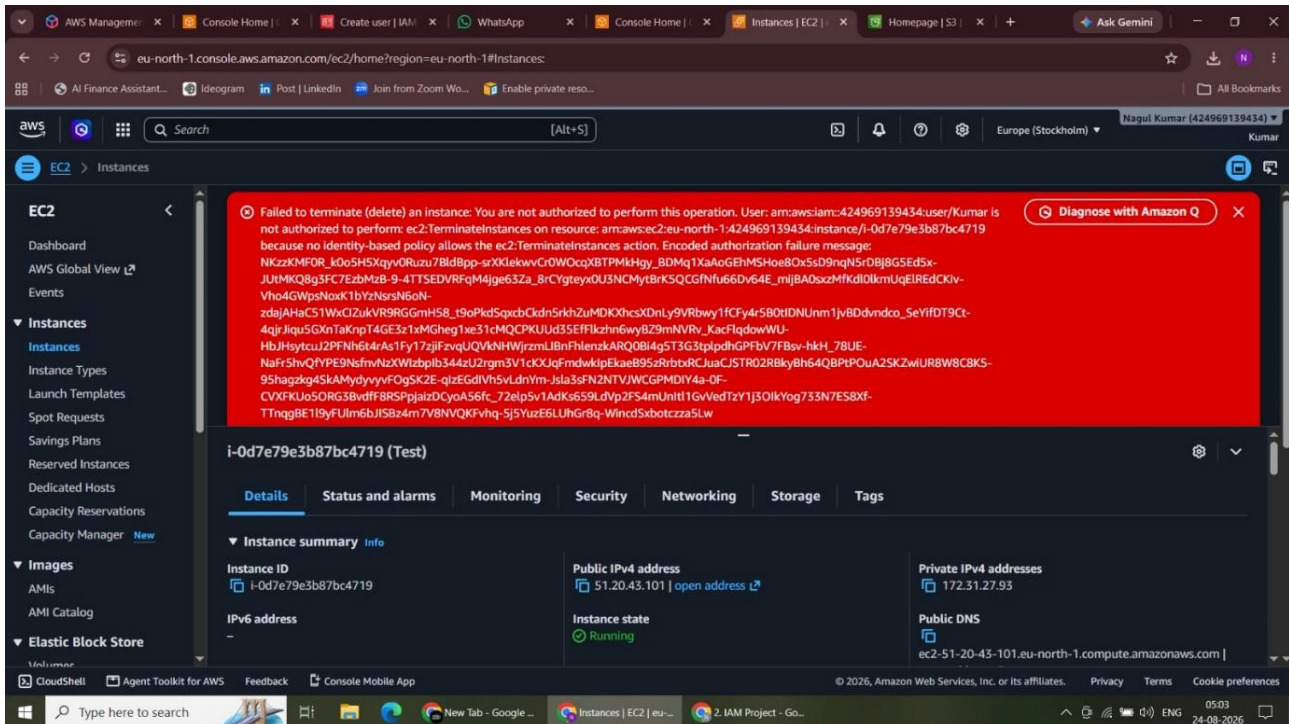


Figure 14. User 2 S3 Bucket Access Test

15. User 2 EC2 Read-Only Validation

Verified that the Storage Admin user can view EC2 resources without full instance management access.

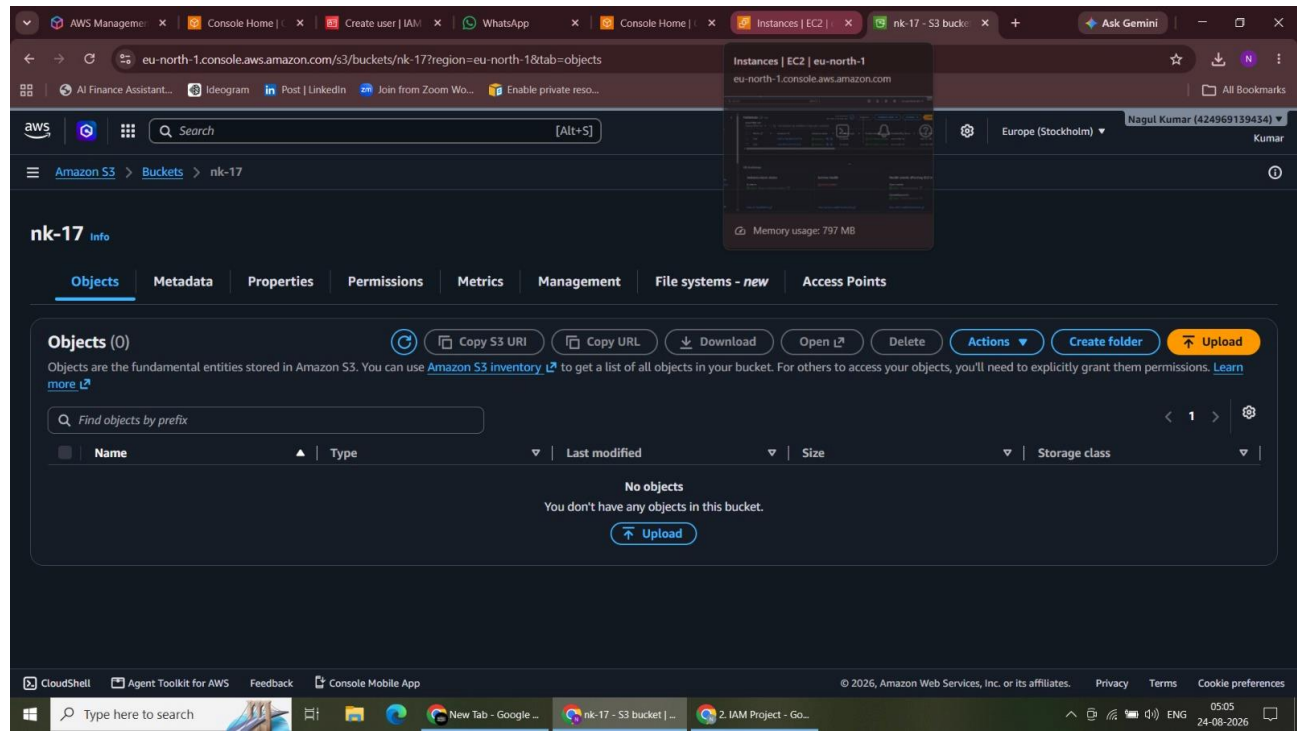


Figure 15. User 2 EC2 Read-Only Validation

16. Restricted EC2 Action Validation

Confirmed that an EC2 action was denied when it was not permitted by the assigned policy.

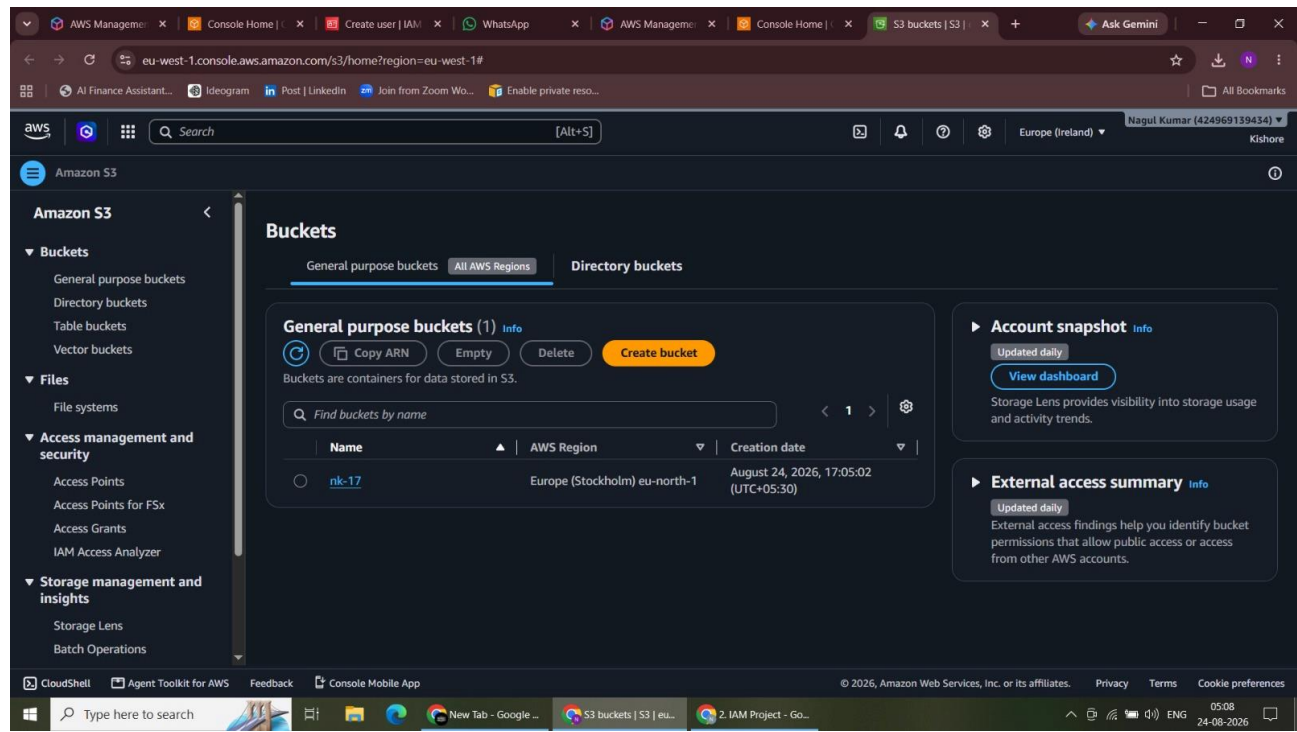


Figure 16. Restricted EC2 Action Validation

17. S3 Bucket Access Verification

Verified access to the S3 bucket and resource details during permission testing.

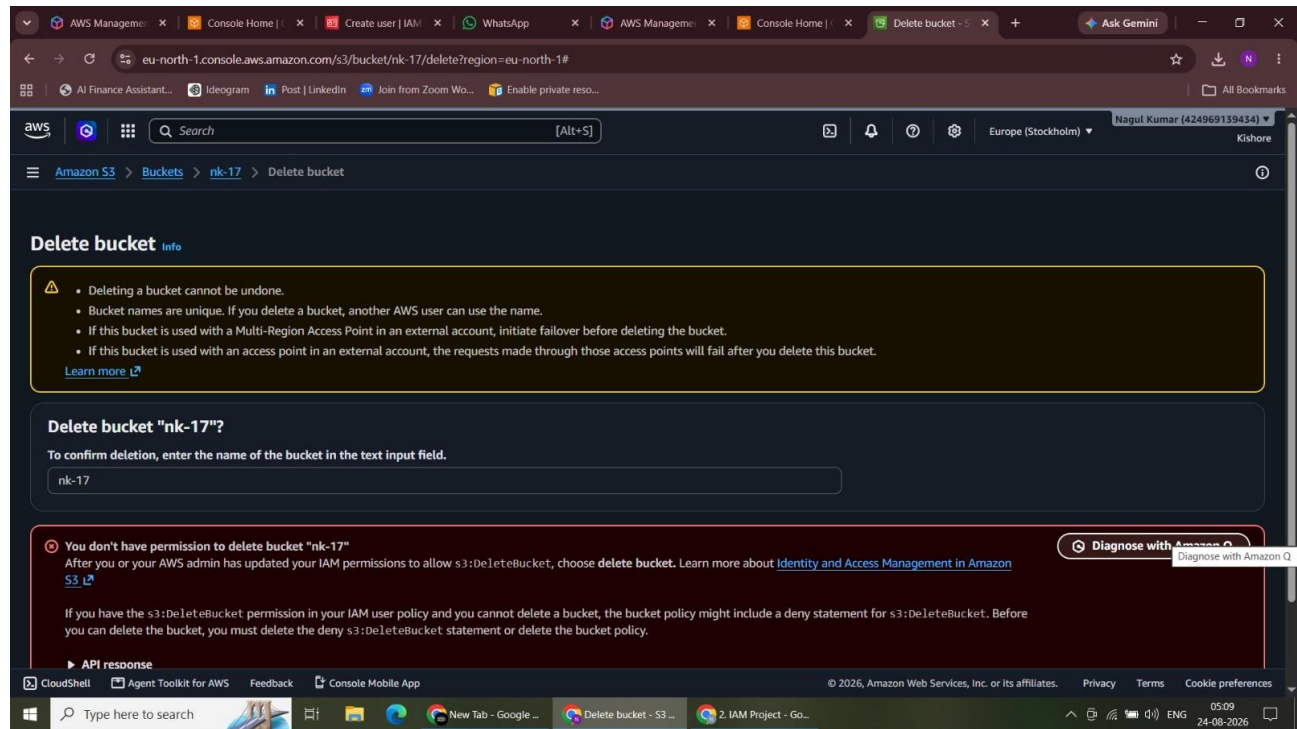


Figure 17. S3 Bucket Access Verification

18. S3 Bucket Listing

Viewed the available S3 bucket list to validate permitted read or list access.

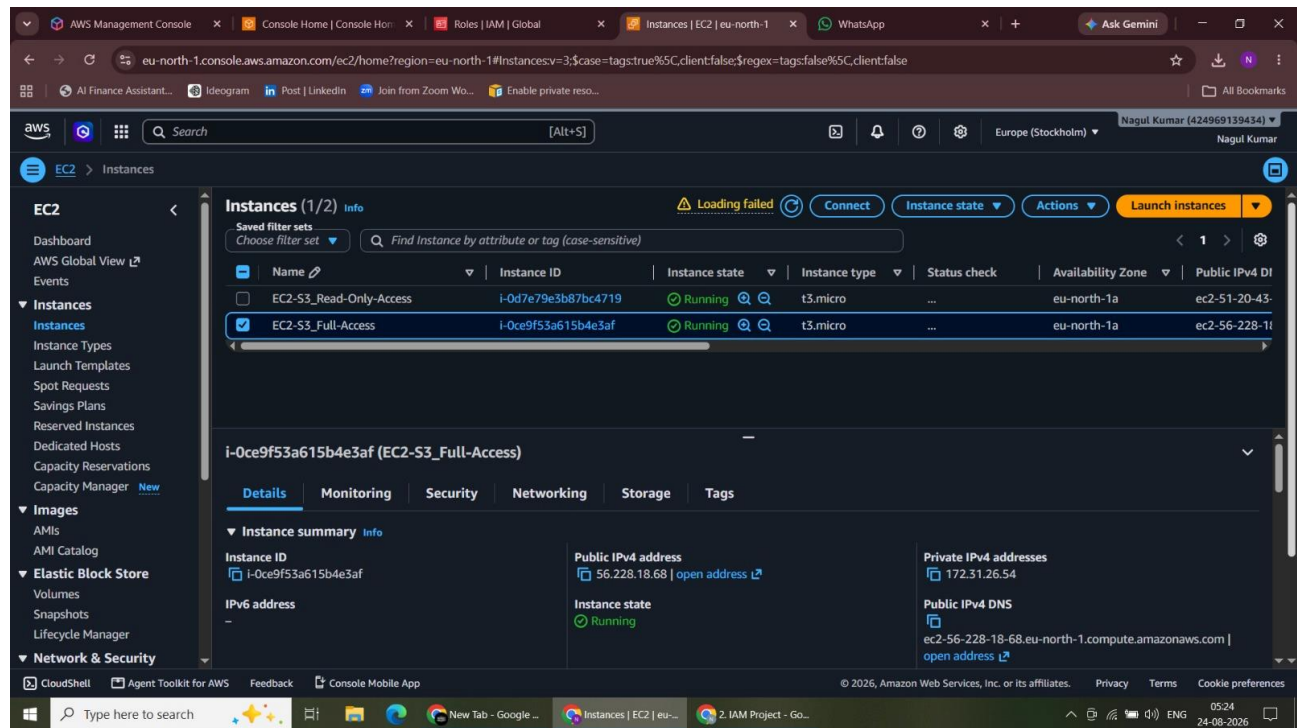


Figure 18. S3 Bucket Listing

19. S3 Bucket Deletion Workflow

Opened the bucket deletion workflow to validate S3 management permissions.

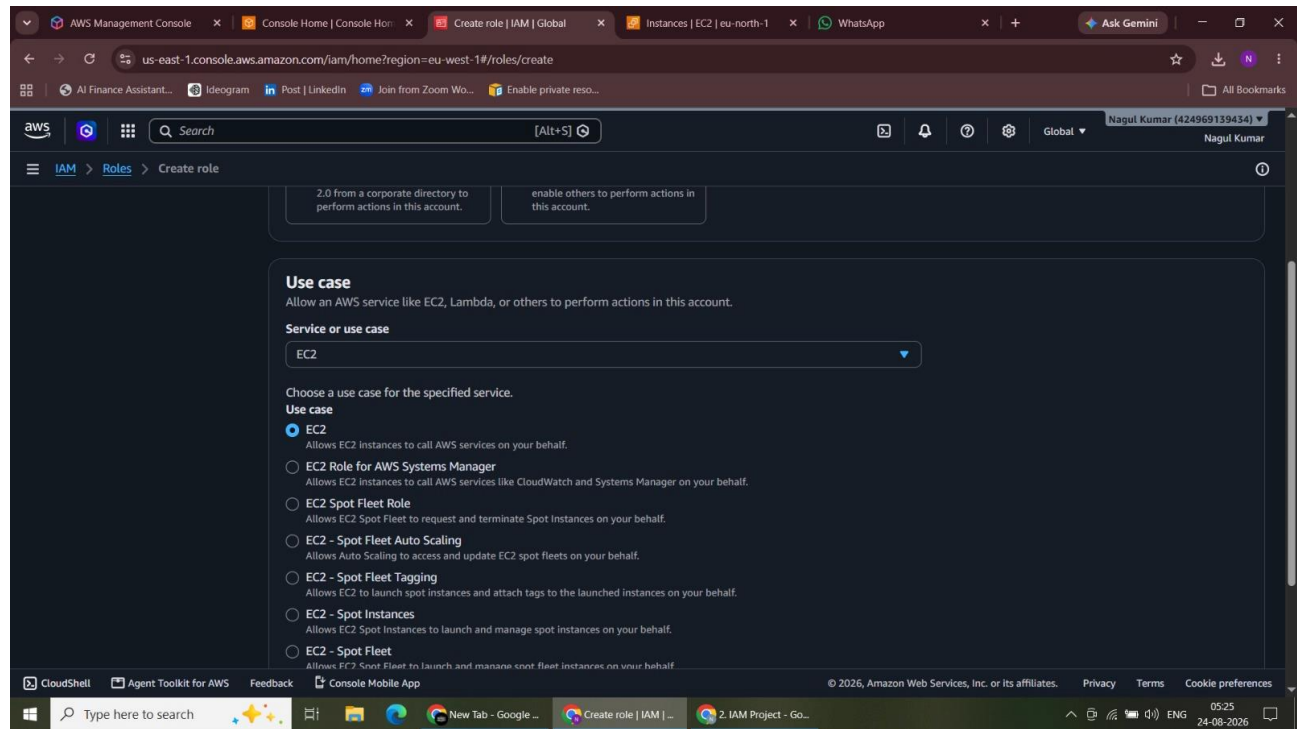


Figure 19. S3 Bucket Deletion Workflow

20. EC2 Instance Status Review

Reviewed the launched EC2 instances before configuring instance-level IAM roles.

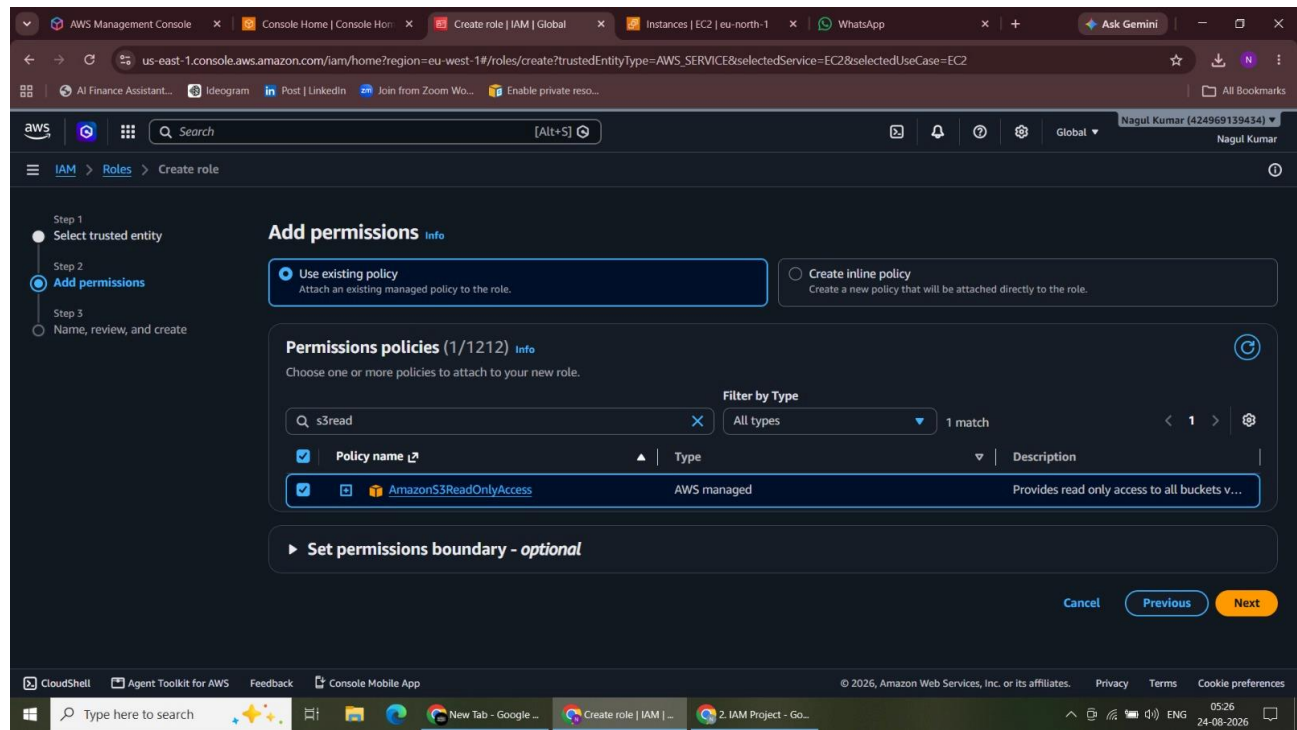


Figure 20. EC2 Instance Status Review

21. Create IAM Role for EC2

Started IAM role creation with EC2 selected as the trusted AWS service.

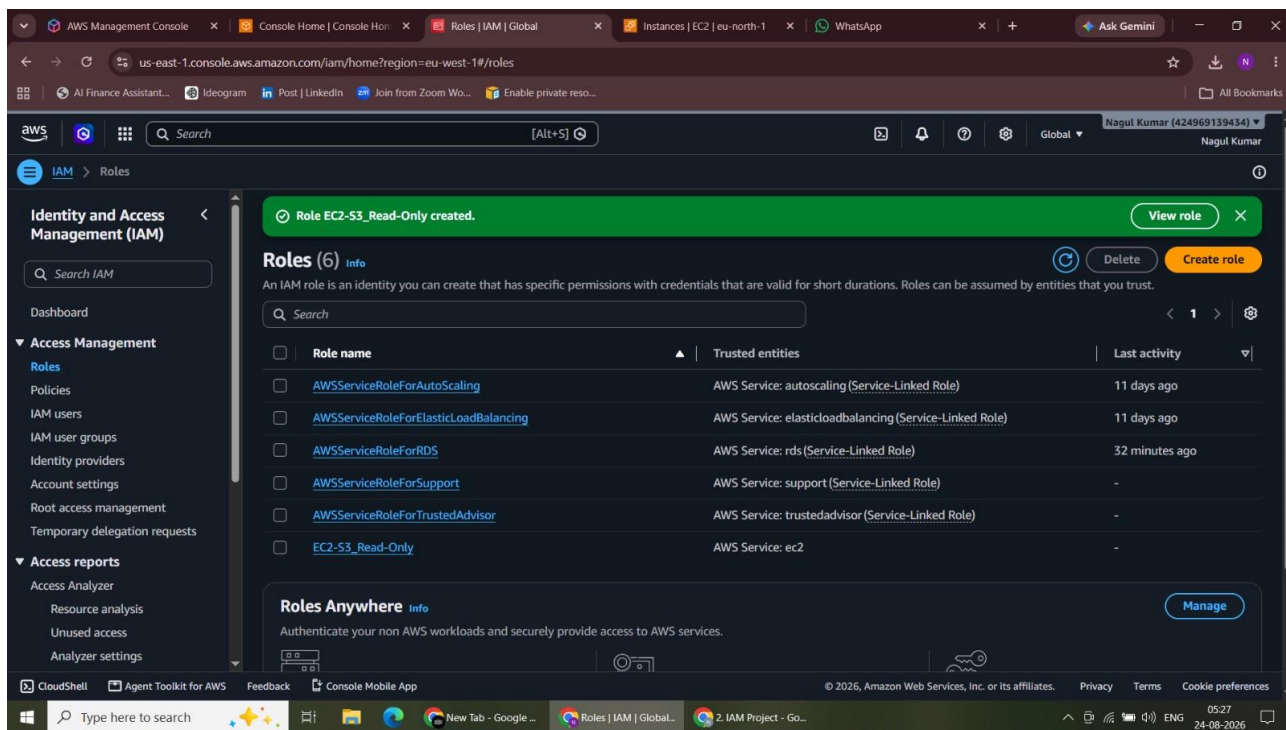


Figure 21. Create IAM Role for EC2

22. IAM Role Permission Configuration

Configured permissions for the EC2 IAM role using the required managed policy.

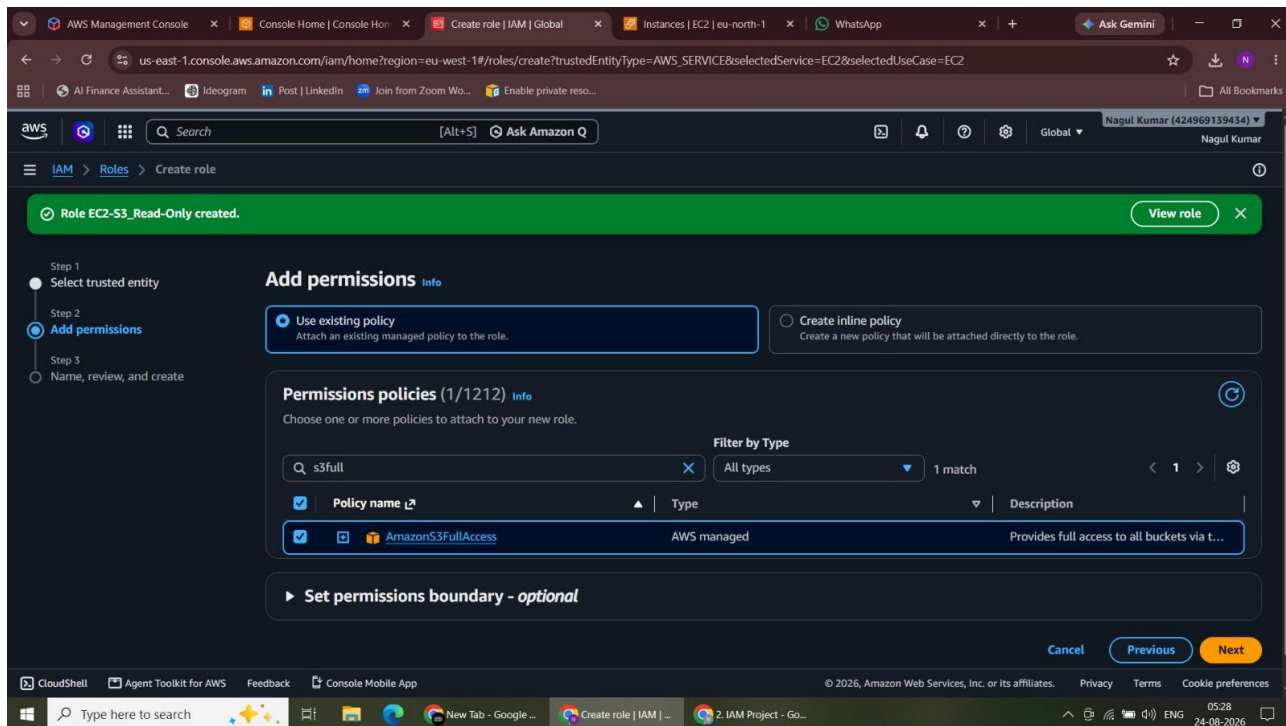


Figure 22. IAM Role Permission Configuration

23. IAM Roles Review

Reviewed the IAM roles created for instance-level access control.

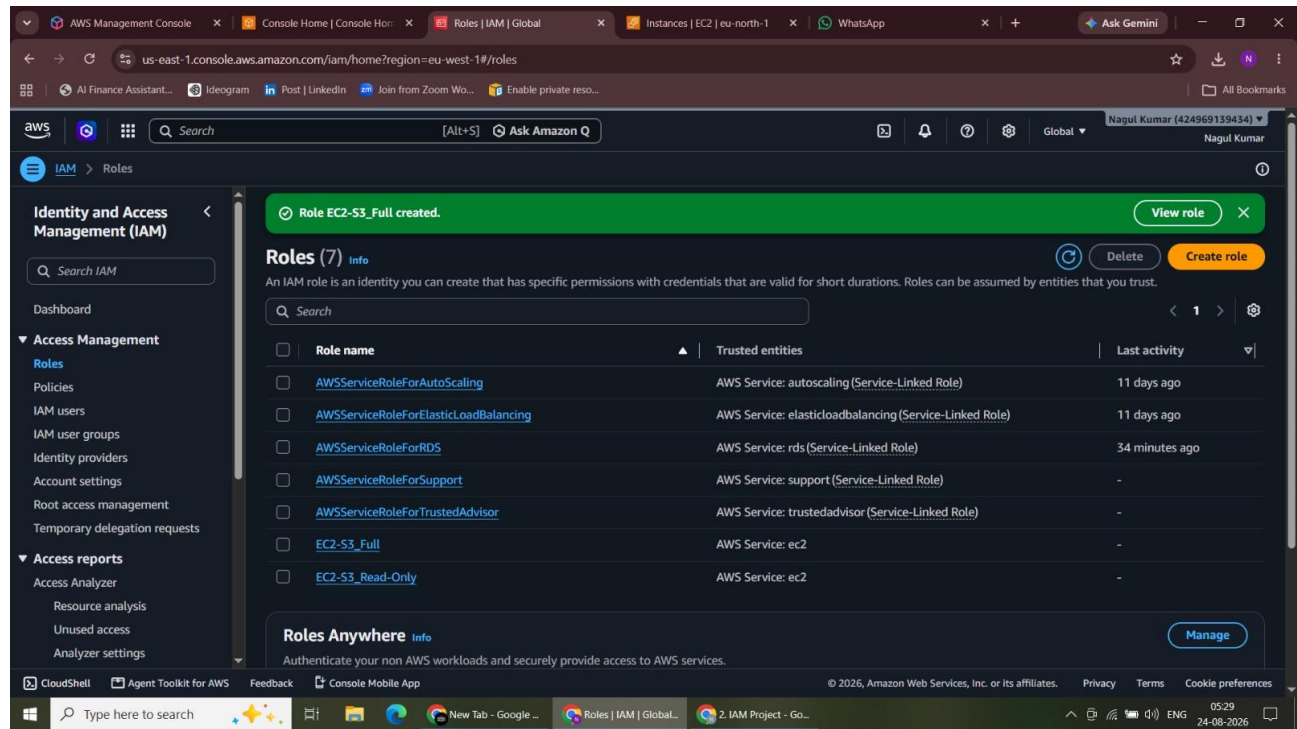


Figure 23. IAM Roles Review

24. S3 Full-Access Policy Selection

Selected the Amazon S3 full-access policy for the corresponding EC2 role.

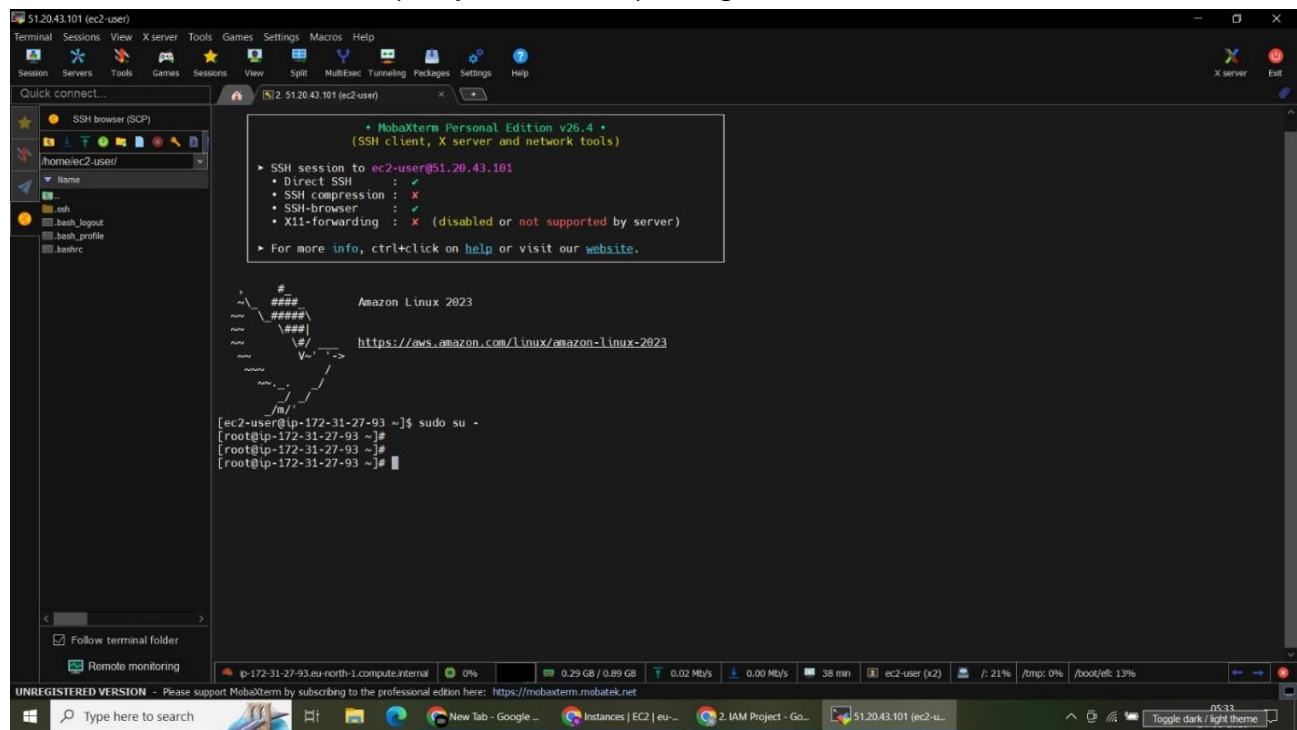


Figure 24. S3 Full-Access Policy Selection

25. IAM Role Creation Confirmation

Confirmed the required IAM roles were available for attachment to EC2 instances.

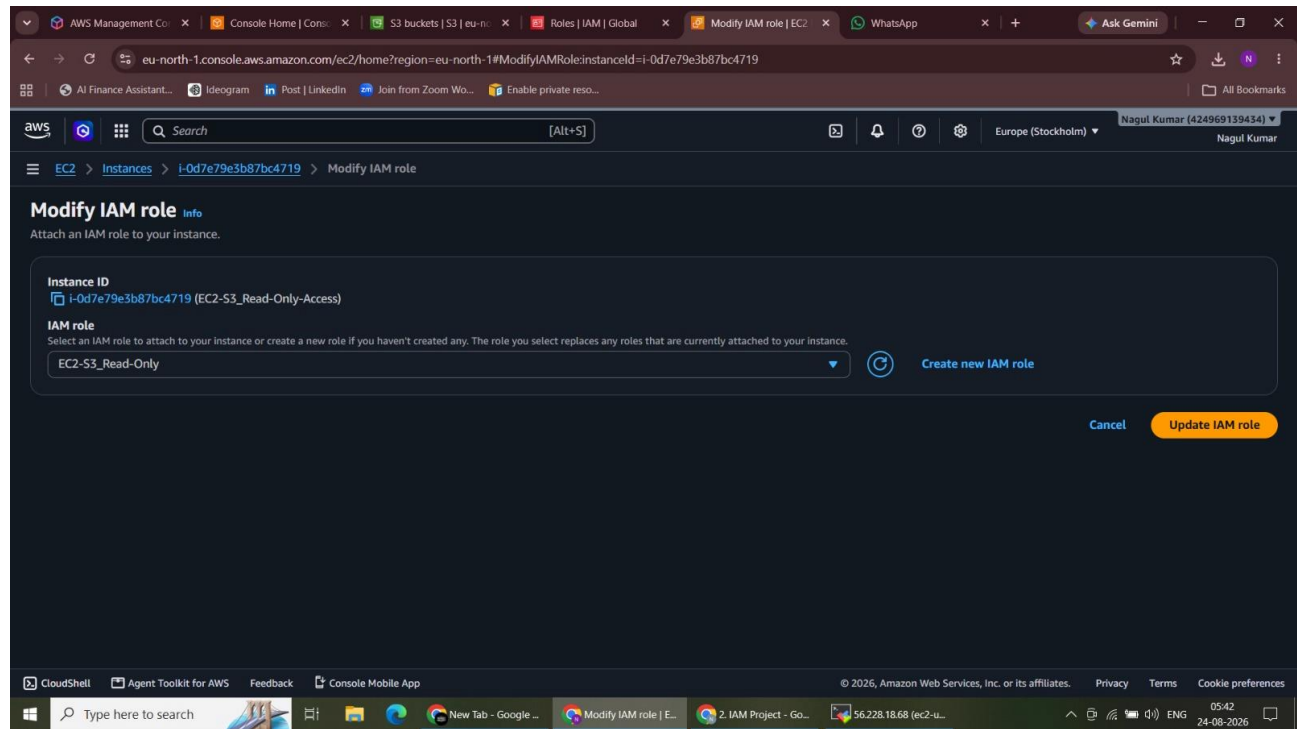


Figure 25. IAM Role Creation Confirmation

26. SSH Connection to EC2 Instance

Connected to the EC2 instance through SSH using MobaXterm for AWS CLI testing.

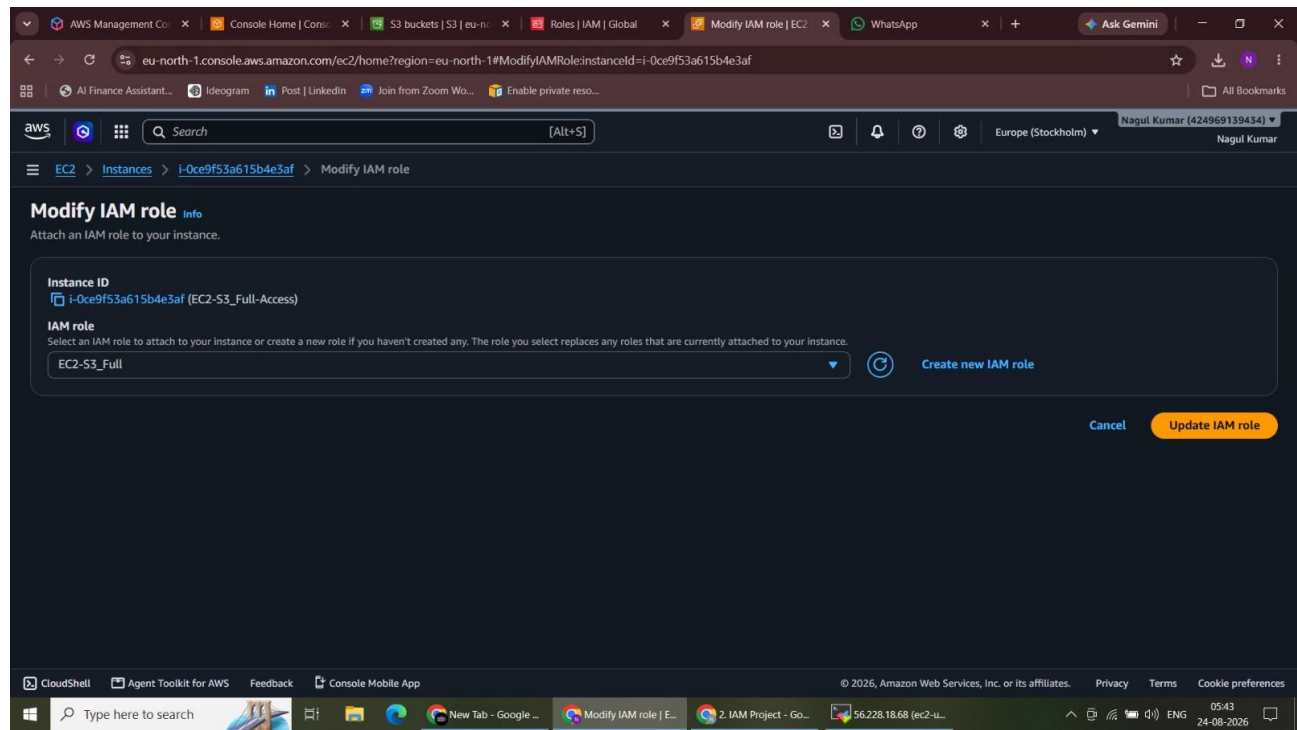
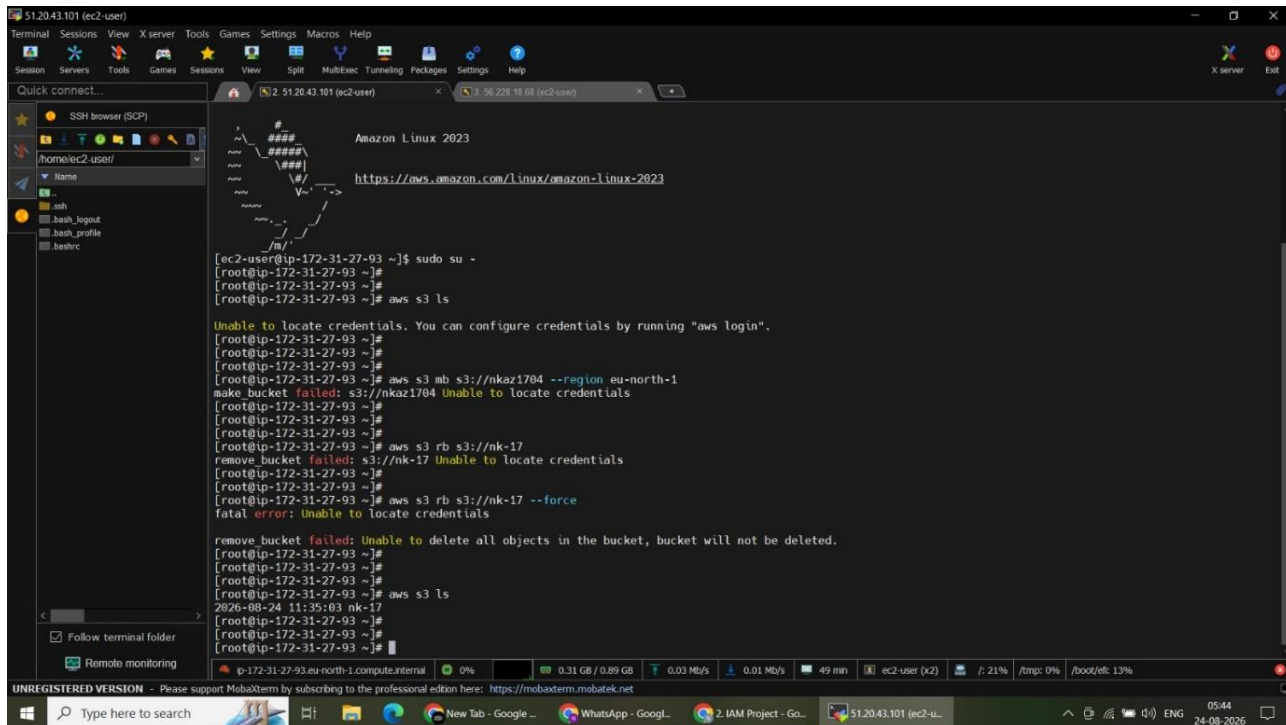


Figure 26. SSH Connection to EC2 Instance

27. Attach S3 Read-Only Role

Attached the S3 read-only IAM role to the EC2-S3_Read-Only instance.



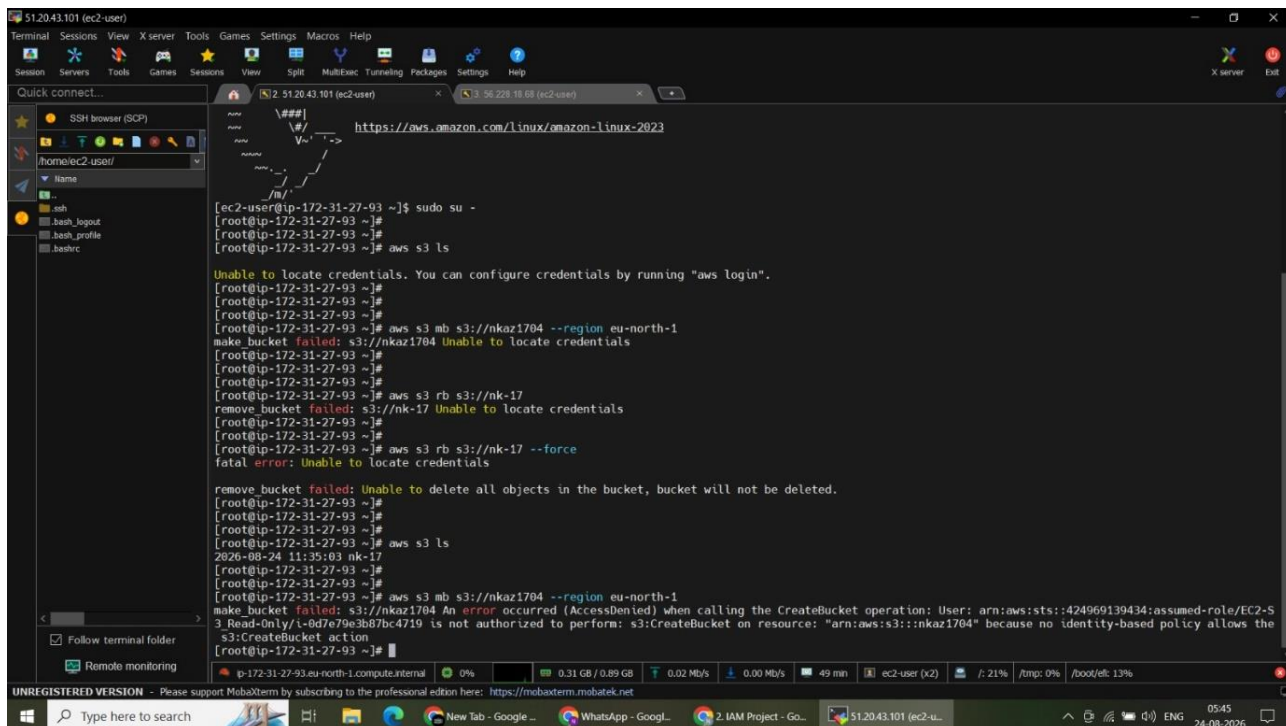
```
Amazon Linux 2023
https://aws.amazon.com/linux/amazon-linux-2023

[ec2-user@ip-172-31-27-93 ~]$ sudo su -
[root@ip-172-31-27-93 ~]#
[root@ip-172-31-27-93 ~]# aws s3 ls
Unable to locate credentials. You can configure credentials by running "aws login".
[root@ip-172-31-27-93 ~]#
[root@ip-172-31-27-93 ~]# aws s3 mb s3://nkaz1704 --region eu-north-1
make_bucket failed: s3://nkaz1704 Unable to locate credentials
[root@ip-172-31-27-93 ~]#
[root@ip-172-31-27-93 ~]# aws s3 rb s3://nk-17
remove_bucket failed: s3://nk-17 Unable to locate credentials
[root@ip-172-31-27-93 ~]#
[root@ip-172-31-27-93 ~]# aws s3 rb s3://nk-17 --force
fatal error: Unable to locate credentials
remove_bucket failed: Unable to delete all objects in the bucket, bucket will not be deleted.
[root@ip-172-31-27-93 ~]#
[root@ip-172-31-27-93 ~]# aws s3 ls
2026-08-24 11:35:03 nk-17
[root@ip-172-31-27-93 ~]#
```

Figure 27. Attach S3 Read-Only Role

28. Attach S3 Full-Access Role

Attached the S3 full-access IAM role to the EC2-S3_Full-Access instance.



```
Amazon Linux 2023
https://aws.amazon.com/linux/amazon-linux-2023

[ec2-user@ip-172-31-27-93 ~]$ sudo su -
[root@ip-172-31-27-93 ~]#
[root@ip-172-31-27-93 ~]# aws s3 ls
Unable to locate credentials. You can configure credentials by running "aws login".
[root@ip-172-31-27-93 ~]#
[root@ip-172-31-27-93 ~]# aws s3 mb s3://nkaz1704 --region eu-north-1
make_bucket failed: s3://nkaz1704 Unable to locate credentials
[root@ip-172-31-27-93 ~]#
[root@ip-172-31-27-93 ~]# aws s3 rb s3://nk-17
remove_bucket failed: s3://nk-17 Unable to locate credentials
[root@ip-172-31-27-93 ~]#
[root@ip-172-31-27-93 ~]# aws s3 rb s3://nk-17 --force
fatal error: Unable to locate credentials
remove_bucket failed: Unable to delete all objects in the bucket, bucket will not be deleted.
[root@ip-172-31-27-93 ~]#
[root@ip-172-31-27-93 ~]# aws s3 ls
2026-08-24 11:35:03 nk-17
[root@ip-172-31-27-93 ~]#
[root@ip-172-31-27-93 ~]# aws s3 mb s3://nkaz1704 --region eu-north-1
make_bucket failed: s3://nkaz1704 An error occurred (AccessDenied) when calling the CreateBucket operation: User: arn:aws:sts::424969139434:assumed-role/EC2-S3-Read-only/I-0d7e79e3b8bc4719 is not authorized to perform: s3:CreateBucket on resource: "arn:aws:s3:::nkaz1704" because no identity-based policy allows the s3:CreateBucket action
[root@ip-172-31-27-93 ~]#
```

Figure 28. Attach S3 Full-Access Role

29. AWS CLI Access Before Role Assignment

Tested AWS CLI commands before effective instance-role access was available and observed missing credentials.

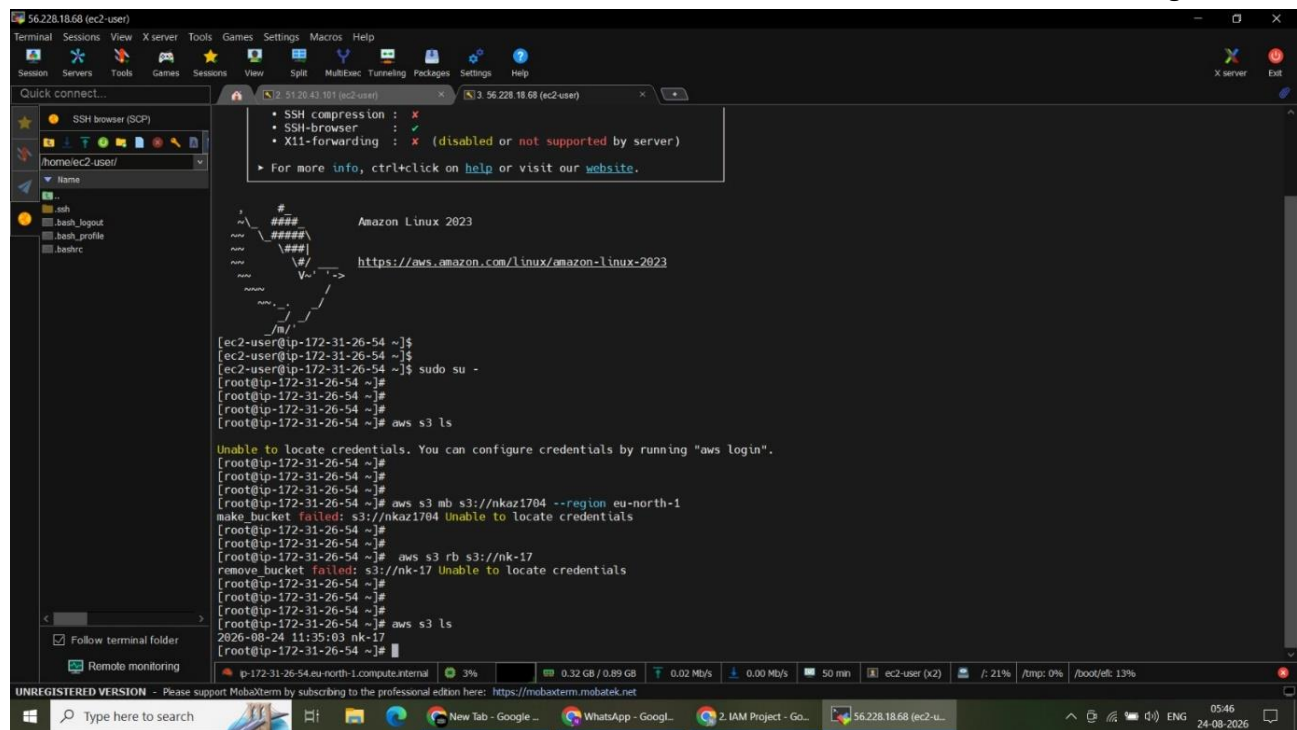


Figure 29. AWS CLI Access Before Role Assignment

30. AWS CLI Permission Test – Instance 1

Validated AWS CLI behavior on the first EC2 instance during IAM role access testing.

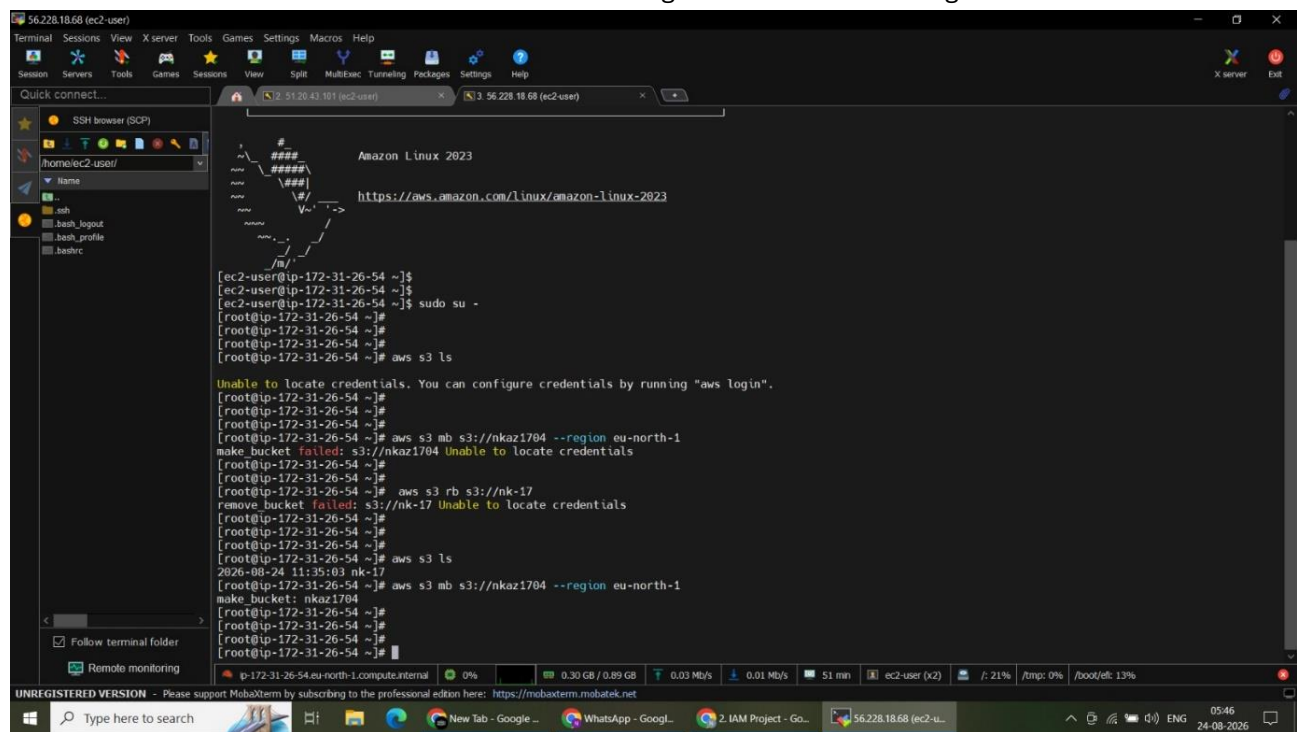
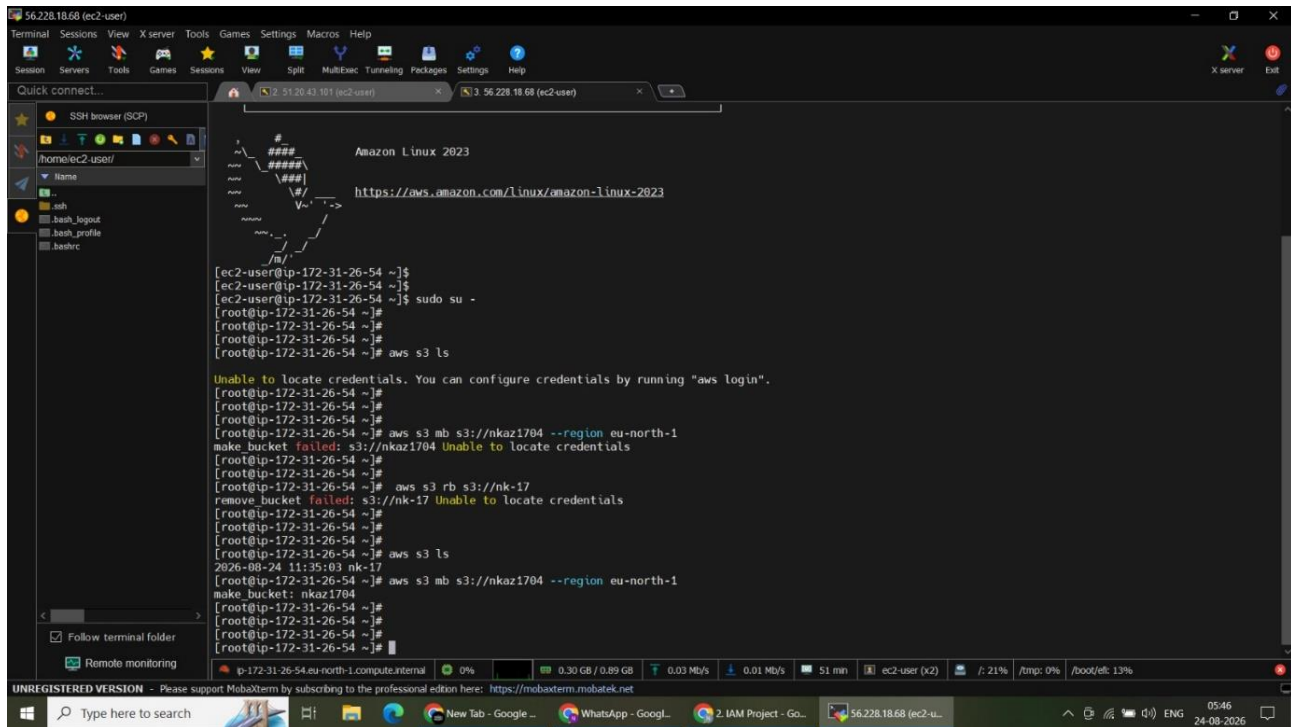


Figure 30. AWS CLI Permission Test – Instance 1

31. AWS CLI Permission Test – Instance 2

Validated AWS CLI behavior on the second EC2 instance during IAM role access testing.



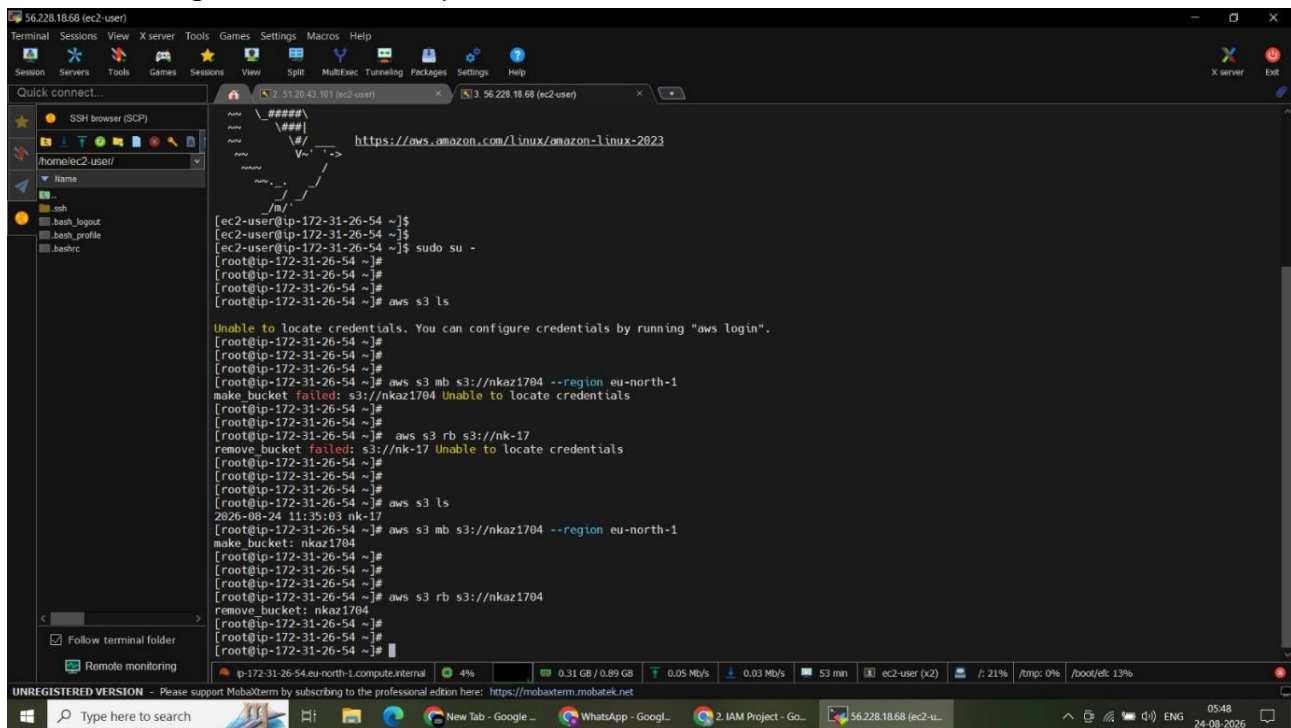
```
Amazon Linux 2023
https://aws.amazon.com/linux/amazon-linux-2023

[ec2-user@ip-172-31-26-54 ~]$
[ec2-user@ip-172-31-26-54 ~]$
[ec2-user@ip-172-31-26-54 ~]$ sudo su -
[root@ip-172-31-26-54 ~]#
[root@ip-172-31-26-54 ~]#
[root@ip-172-31-26-54 ~]# aws s3 ls
[ec2-user@ip-172-31-26-54 ~]$
[ec2-user@ip-172-31-26-54 ~]$
[ec2-user@ip-172-31-26-54 ~]$ sudo su -
[root@ip-172-31-26-54 ~]#
[root@ip-172-31-26-54 ~]#
[root@ip-172-31-26-54 ~]# aws s3 ls
Unable to locate credentials. You can configure credentials by running "aws login".
[root@ip-172-31-26-54 ~]#
[root@ip-172-31-26-54 ~]#
[root@ip-172-31-26-54 ~]# aws s3 mb s3://nkaz1704 --region eu-north-1
make_bucket failed: s3://nkaz1704 Unable to locate credentials
[root@ip-172-31-26-54 ~]#
[root@ip-172-31-26-54 ~]# aws s3 rb s3://nk-17
remove_bucket failed: s3://nk-17 Unable to locate credentials
[root@ip-172-31-26-54 ~]#
[root@ip-172-31-26-54 ~]#
[root@ip-172-31-26-54 ~]# aws s3 ls
2026-08-24 11:35:03 nk-17
[root@ip-172-31-26-54 ~]# aws s3 mb s3://nkaz1704 --region eu-north-1
make_bucket: nkaz1704
[root@ip-172-31-26-54 ~]#
[root@ip-172-31-26-54 ~]#
[root@ip-172-31-26-54 ~]# aws s3 ls
2026-08-24 11:35:03 nk-17
[root@ip-172-31-26-54 ~]# aws s3 mb s3://nkaz1704 --region eu-north-1
make_bucket: nkaz1704
[root@ip-172-31-26-54 ~]#
[root@ip-172-31-26-54 ~]#
[root@ip-172-31-26-54 ~]# aws s3 rb s3://nkaz1704
remove_bucket: nkaz1704
[root@ip-172-31-26-54 ~]#
[root@ip-172-31-26-54 ~]#
[root@ip-172-31-26-54 ~]#
```

Figure 31. AWS CLI Permission Test – Instance 2

32. AWS CLI Permission Test – Continued

Continued validating S3 commands and permissions from the EC2 instance terminal.



```
Amazon Linux 2023
https://aws.amazon.com/linux/amazon-linux-2023

[ec2-user@ip-172-31-26-54 ~]$
[ec2-user@ip-172-31-26-54 ~]$
[ec2-user@ip-172-31-26-54 ~]$ sudo su -
[root@ip-172-31-26-54 ~]#
[root@ip-172-31-26-54 ~]#
[root@ip-172-31-26-54 ~]# aws s3 ls
Unable to locate credentials. You can configure credentials by running "aws login".
[root@ip-172-31-26-54 ~]#
[root@ip-172-31-26-54 ~]#
[root@ip-172-31-26-54 ~]# aws s3 mb s3://nkaz1704 --region eu-north-1
make_bucket failed: s3://nkaz1704 Unable to locate credentials
[root@ip-172-31-26-54 ~]#
[root@ip-172-31-26-54 ~]#
[root@ip-172-31-26-54 ~]# aws s3 rb s3://nk-17
remove_bucket failed: s3://nk-17 Unable to locate credentials
[root@ip-172-31-26-54 ~]#
[root@ip-172-31-26-54 ~]#
[root@ip-172-31-26-54 ~]# aws s3 ls
2026-08-24 11:35:03 nk-17
[root@ip-172-31-26-54 ~]# aws s3 mb s3://nkaz1704 --region eu-north-1
make_bucket: nkaz1704
[root@ip-172-31-26-54 ~]#
[root@ip-172-31-26-54 ~]#
[root@ip-172-31-26-54 ~]# aws s3 rb s3://nkaz1704
remove_bucket: nkaz1704
[root@ip-172-31-26-54 ~]#
[root@ip-172-31-26-54 ~]#
[root@ip-172-31-26-54 ~]#
```

Figure 32. AWS CLI Permission Test – Continued

33. Final S3 Bucket Verification

Verified the resulting S3 bucket state after successful role-based AWS CLI operations.

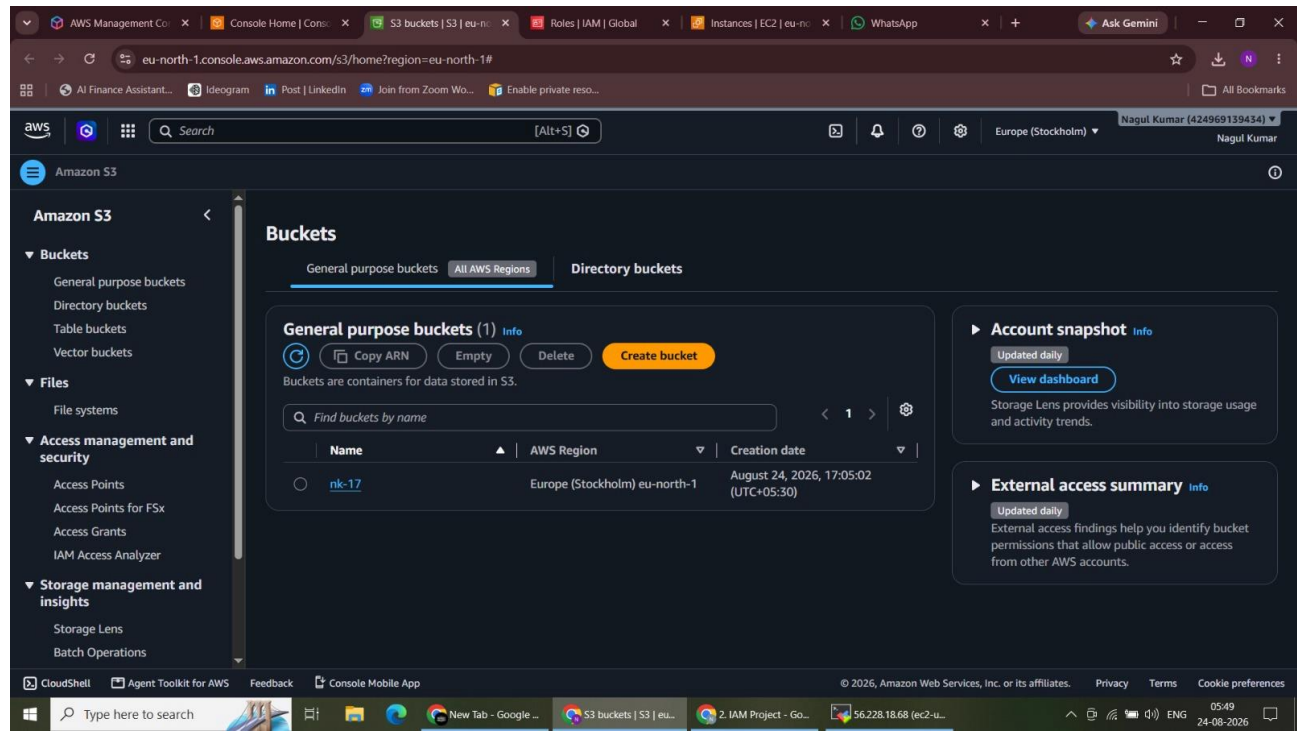


Figure 33. Final S3 Bucket Verification

7. Validation Summary

The screenshots demonstrate the complete IAM implementation, from creating user groups and IAM users to validating policy-based access restrictions and configuring EC2 instance roles for controlled S3 access.

8. Key Learning Outcomes

- IAM users and user groups for centralized permission management.
- AWS managed policies and least-privilege access validation.
- EC2 full access versus EC2 read-only permission behavior.
- S3 full access versus S3 read-only permission behavior.
- IAM roles for granting temporary permissions to EC2 instances.
- AWS CLI access to S3 through attached instance roles.
- SSH-based EC2 administration using MobaXterm.
- Validation of both allowed and denied AWS operations.

9. Conclusion

The project successfully implemented and validated a role-based AWS access-control model using IAM users, groups, policies, and EC2 instance roles. The final testing demonstrated how AWS permissions allow or deny specific EC2 and S3 operations at both the user and instance levels.